

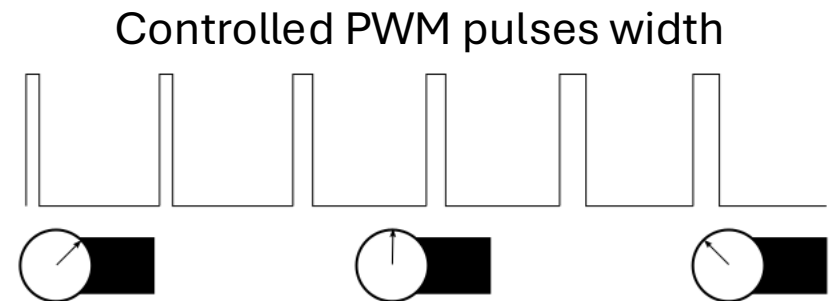
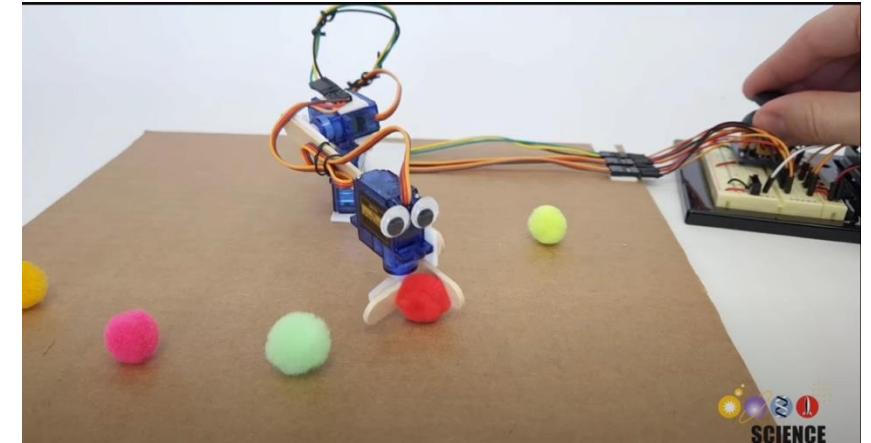
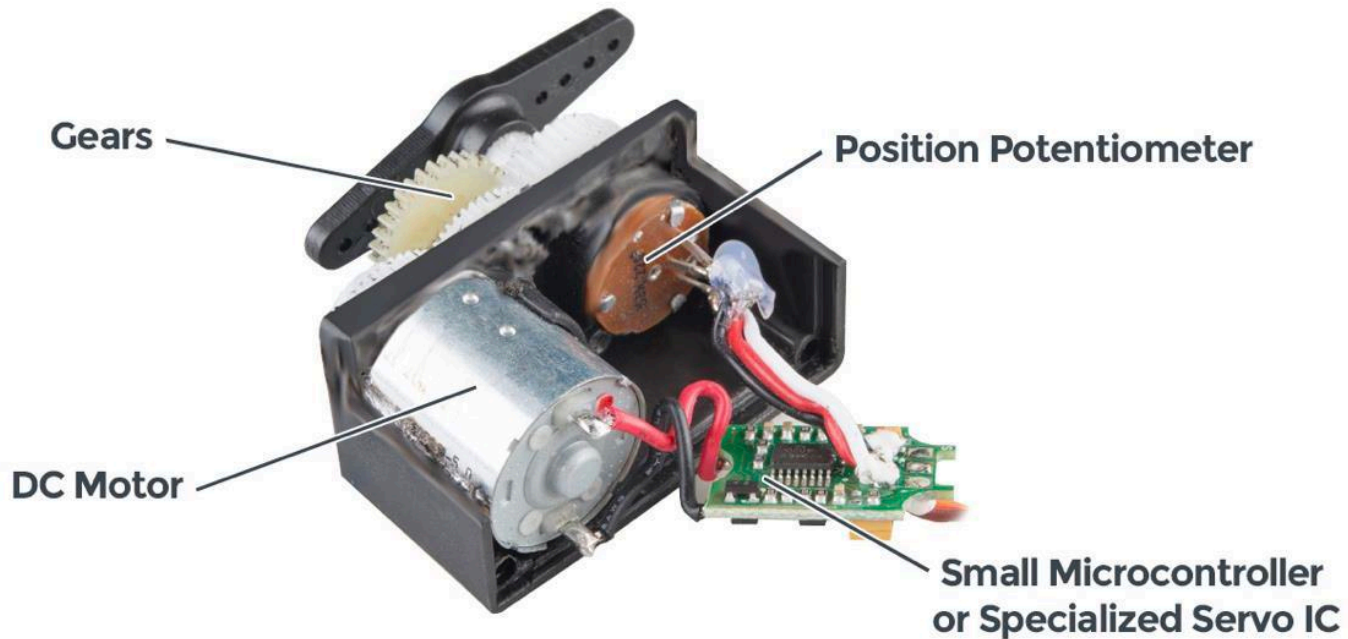
EE 186: Introduction to Embedded Systems

Lecture 4

Zerina Kapetanovic

Component of the Day....

Servos!

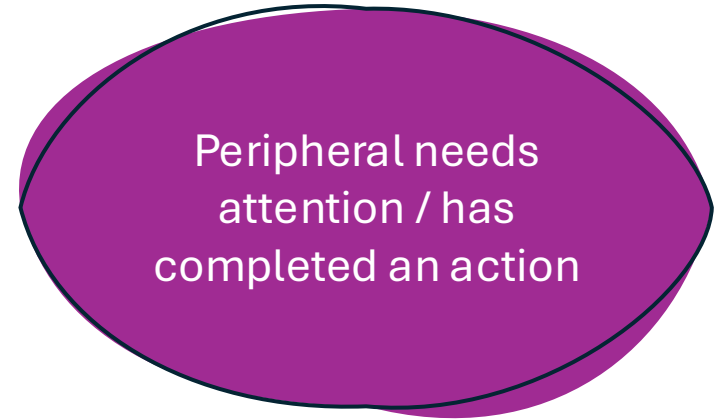


Today we are going to learn about...

- Interrupts
 - Polling vs. Interrupts
 - Vector Tables
 - Stacking and Unstacking
 - Interrupt Control Registers
 - Interrupt Flow Control
 - Interrupt Priorities
- Nested Vector Interrupt Controller (NVIC)

Asynchronous Events

- During a program execution, there can be many “events”



How do we know that an event has occurred?

Detecting an event has occurred

Polling

- Repeatedly poll the app/processor/peripherals
- When an event occurs, detect this via a poll and take action.

vs.

Interrupts

- Let the app/processors/peripheral notify us instead
- Take action when a notification occurs (or shortly after)

Polling Drawbacks

Example: Push button to turn on LED

```
                                mov r0, #0x4           % push button MMIO address
                                mov r1, #0x5           % LED MMIO address
loop:                          ldr r2, [r0, #0]        % read value from switch [1 cycle]
                                str r2, [r1, #0]        % save value to LED [1 cycle]
                                b loop                 % repeat these steps [1 cycle]
```

- Software is continuously looping, polling, and reacting
- Responses are fast, average 3 cycles!
- Doesn't do anything else useful

Polling Drawbacks

Example: Push button to turn on LED and run “do some work”

```
                                mov r0, #0x4           % push button MMIO address
                                mov r1, #0x5           % LED MMIO address
loop:                          ldr r2, [r0, #0]         % read value from switch [1 cycle]
                                str r2, [r1, #0]         % save value to LED [1 cycle]
                                bl do_some_work         % do some work [100 cycles]
                                b loop                  % repeat these steps [1 cycle]
```

- Added function that “does some work” ... 100 more cycles
- Whenever we are “doing some work” we are not polling the push button
- Latency increased to ~103 cycles for polling!

Polling scales very poorly!

```

                                mov r0, #0x4           % push button MMIO address
                                mov r1, #0x5           % LED MMIO address
loop:                          ldr r2, [r0, #0]        % read value from switch [1 cycle]
                                str r2, [r1, #0]        % save value to LED [1 cycle]
                                bl do_some_work         % do some work [100 cycles]
                                bl do_other_work        $ do other work [100 cycles]

                                ...

                                bl even_more_work      % even more work [10,000 cycles]
                                b loop                  % repeat these steps [1 cycle]
```

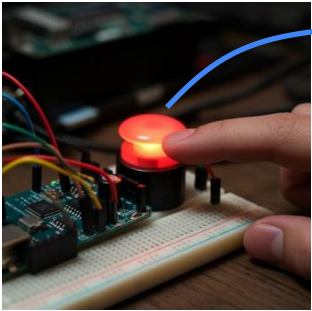


Your system becomes less responsive to external events
(e.g., push button press)



Do better with interrupts!

An interrupt informs a program of an **event** and **breaks execution** flow



An **event** occurs that causes the CPU to stop executing the current program

Begin executing a special piece of code (**aka Interrupt Service Routine**)

The **Interrupt Service Routine (ISR)** does some small amount of work and then resumes the interrupted program.

Types of Interrupts

Instruction / System Faults

Examples

- Illegal/unimplemented instruction
- Divide by 0

Names

- System Exception
- Internal Interrupt

External / Non-instruction

Examples

- Power failure
- Button push
- SPI or I2C Communication Bus

Names

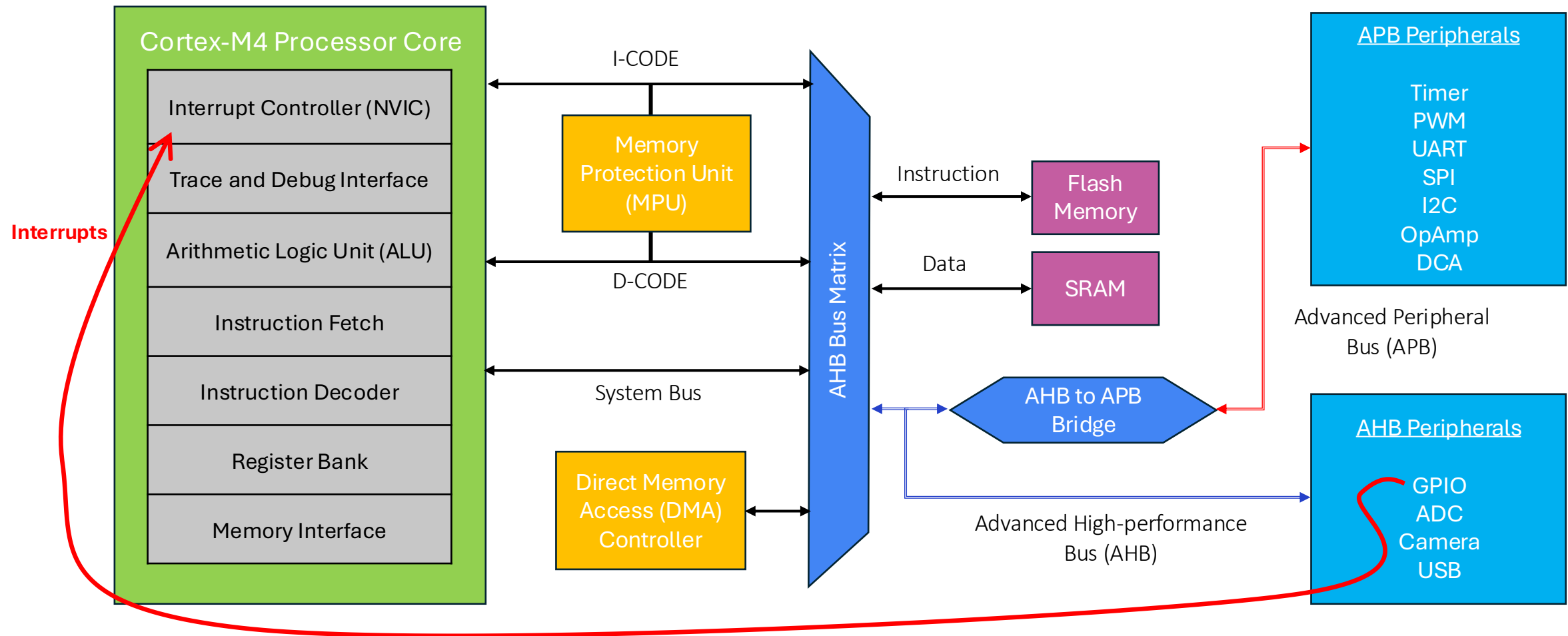
- Interrupt
- External Interrupt

Interrupt Overview

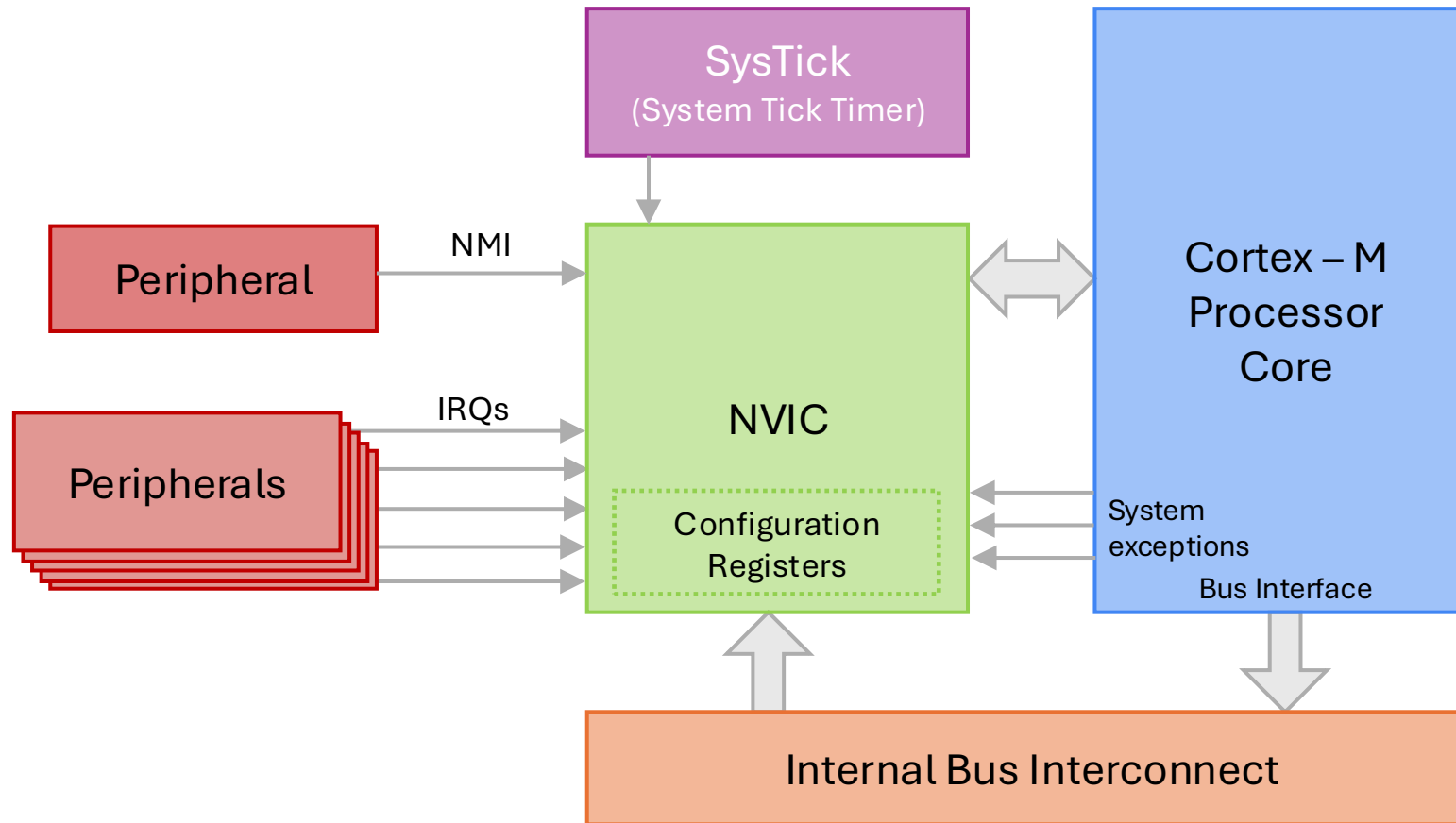
- 1 Interrupt signal is generated (hardware, sometimes firmware)
- 2 Determine which interrupt fired and what code to execute
- 3 Halt *main* code execution and run **Interrupt Service Routine (ISR)**
- 4 Return processor to original state and continue old program

30 second
brainstorm

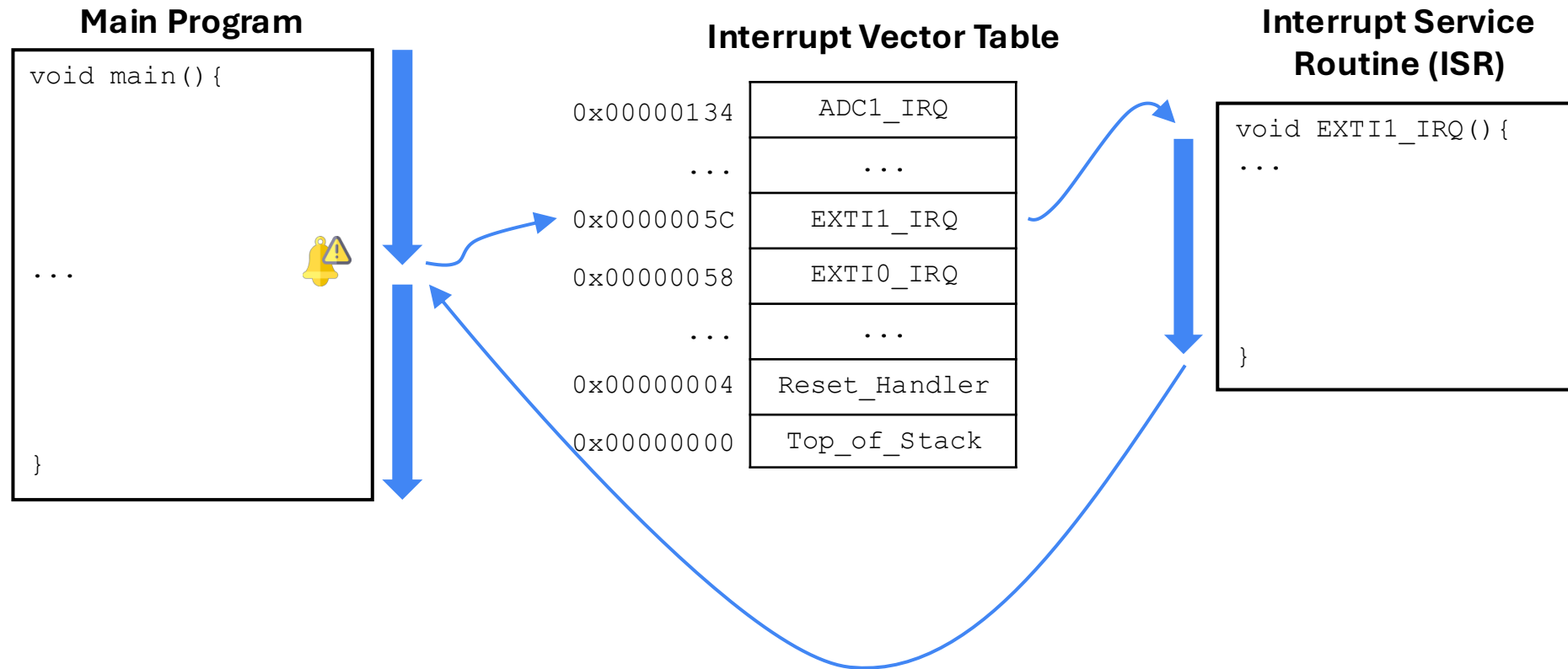
How do we get an interrupt from a peripheral to the core?



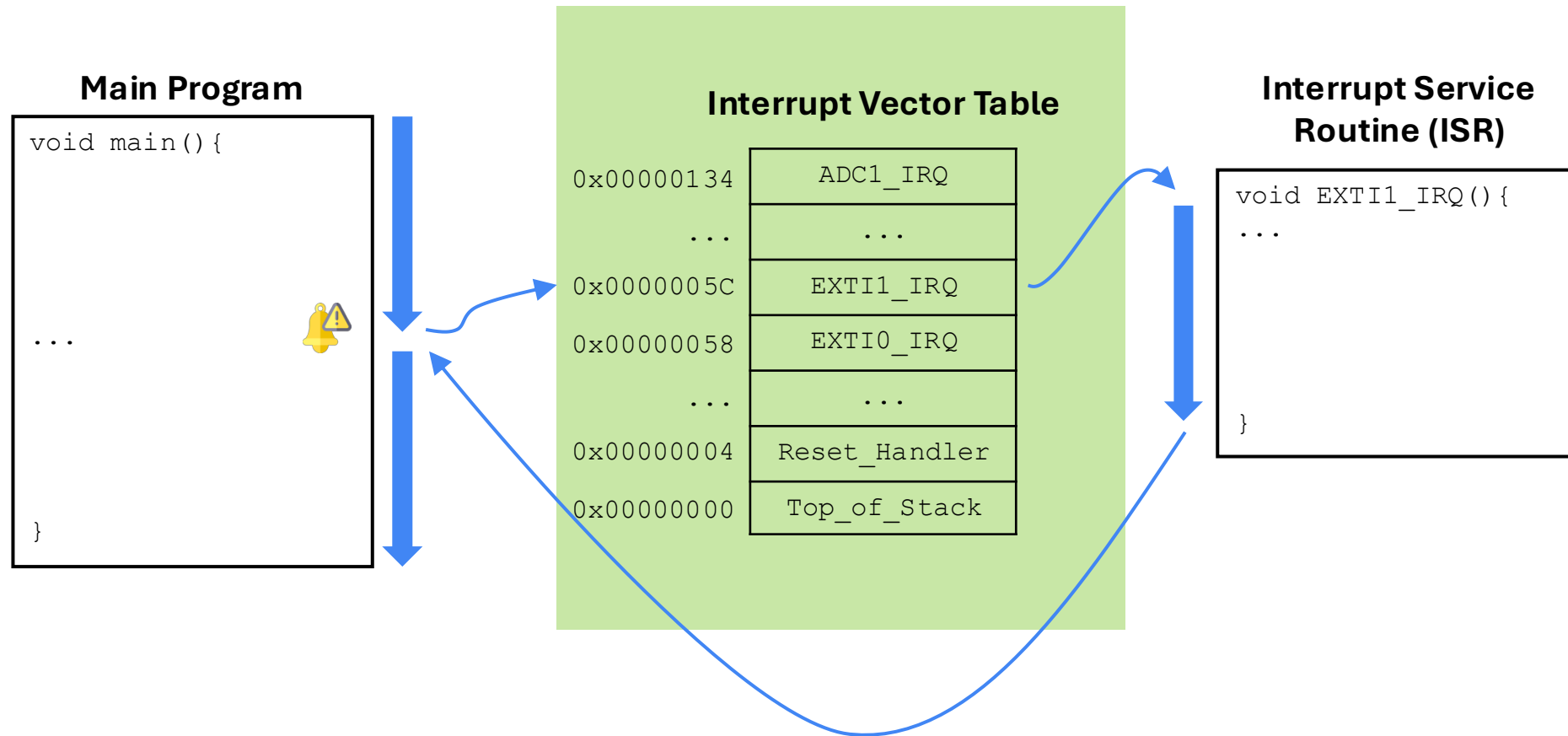
Interrupt Overview



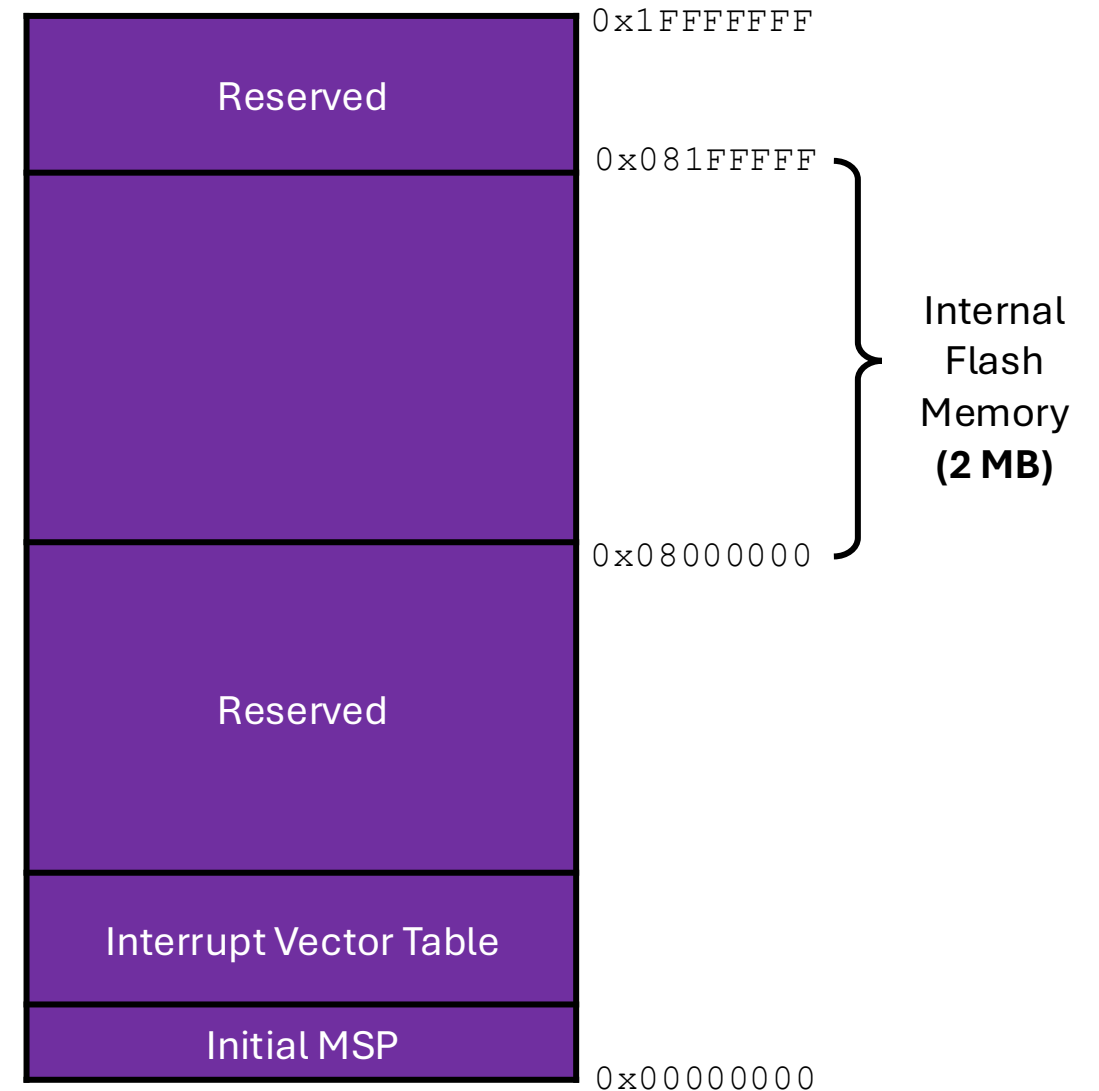
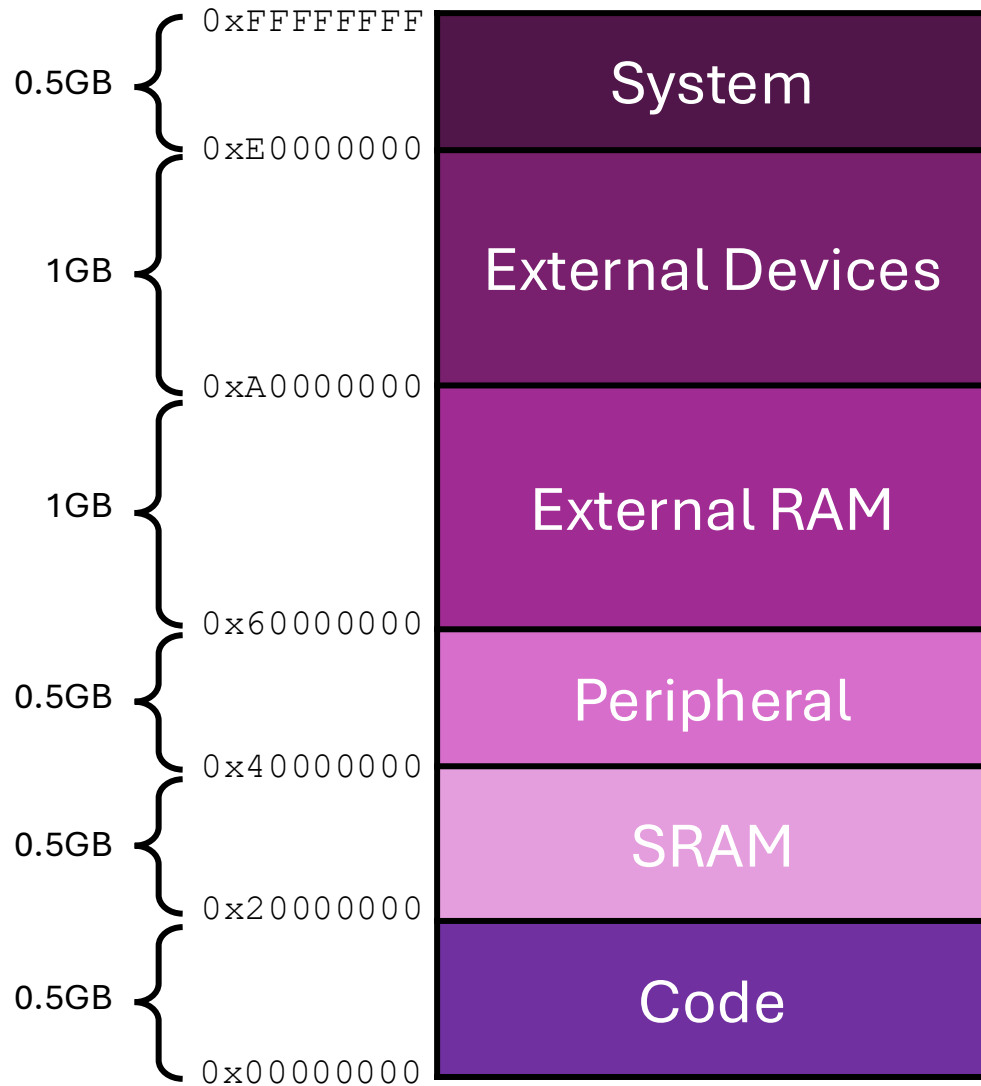
Interrupt Overview



Interrupt Overview



Instruction Memory



ISR Vector Table

The vector table is a list of **function addresses** stored at the beginning of memory (a list of pointers)

The functions are called Interrupt Service Routines (ISRs)

- C/Assembly code that does “some task”

0x00000000 holds the value of the **Stack Pointer**

PC is initialized to 0x00000004

System
Exceptions

Interrupt Number	Memory Address	Memory Contents (32 bits)	
13	0x00000074	DMA1_Channel3_IRQHandler	
12	0x00000070	DMA1_Channel2_IRQHandler	
11	0x0000006C	DMA1_Channel1_IRQHandler	
10	0x00000068	EXTI4_IRQHandler	
9	0x00000064	EXTI3_IRQHandler	
8	0x00000060	EXTI2_IRQHandler	
7	0x0000005C	EXTI1_IRQHandler	
6	0x00000058	EXTI0_IRQHandler	
5	0x00000054	RCC_IRQHandler	
4	0x00000050	FLASH_IRQHandler	
3	0x0000004C	RTC_WKUP_IRQHandler	
2	0x00000048	TAMPER_STAMP_IRQHandler	
1	0x00000044	PVD_IRQHandler	
0	0x00000040	WWDG_IRQHandler	
-1	0x0000003C	SysTick_Handler	
-2	0x00000038	PendSV_Handler	
-3	0x00000034	Reserved	
-4	0x00000030	DebugMon_Handler	
-5	0x0000002C	SVC_Handler	
-6	0x00000028	Reserved	
-7	0x00000024	Reserved	
-8	0x00000020	Reserved	
-9	0x0000001C	Reserved	
-10	0x00000018	UsageFault_Handler	
-11	0x00000014	BusFault_Handler	
-12	0x00000010	MemManage_Handler	
-13	0x0000000C	HardFault_Handler	
-14	0x00000008	NMI_Handler	
	0x00000004	Reset_Handler	Value to initialize program counter (PC)
	0x00000000	Top_of_Stack	Value to initialize stack pointer (SP)

Looking up IRQs

Interrupt signals sent to the NVIC are called **Interrupt Requests (IRQs)**

Each IRQ has an assigned number

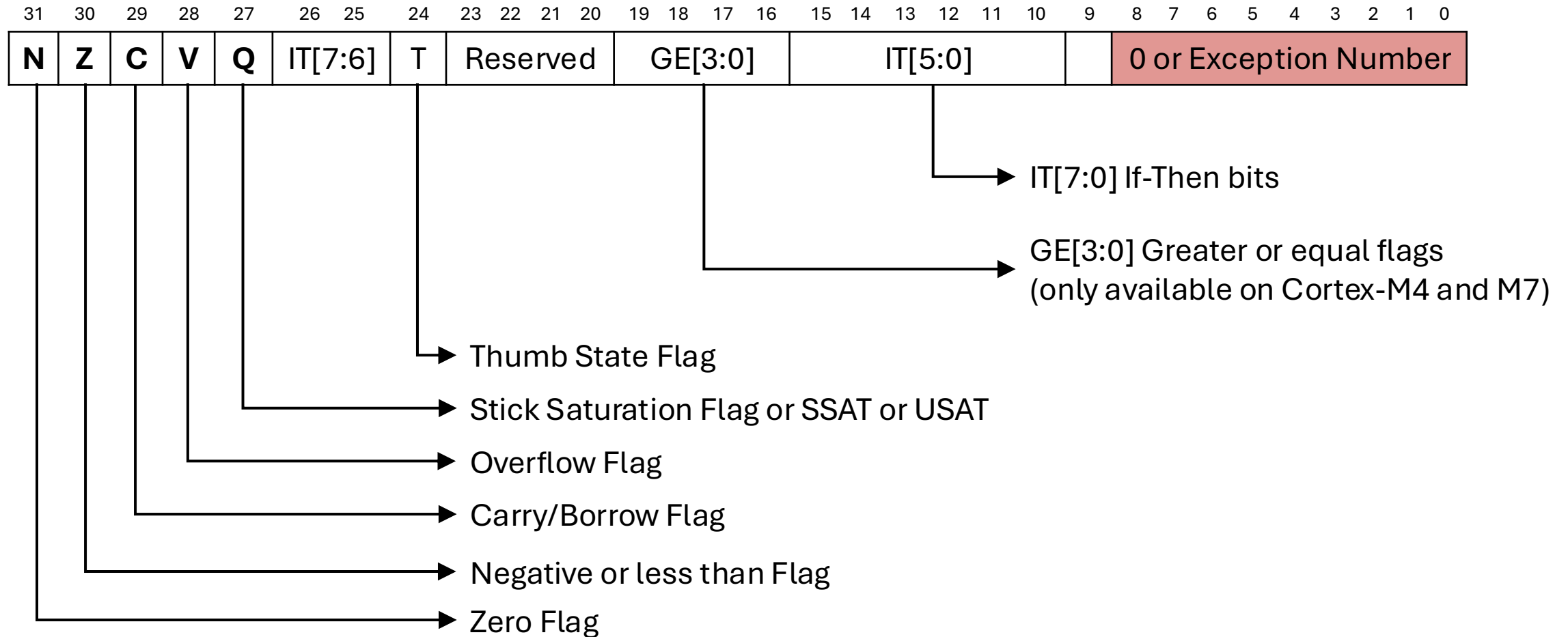
- ARM supports up to 256 IRQs
- STM32L4R has 96 + 16

The first 16 interrupts are called **System Exception** and are defined by the ARM CMSIS

When the NVIC receives an IRQ it uses the **Interrupt Number** to loop up the memory address of the function in the vector table

System Exceptions	13	0x00000074	DMA1_Channel3_IRQHandler
	12	0x00000070	DMA1_Channel2_IRQHandler
	11	0x0000006C	DMA1_Channel1_IRQHandler
	10	0x00000068	EXTI4_IRQHandler
	9	0x00000064	EXTI3_IRQHandler
	8	0x00000060	EXTI2_IRQHandler
	7	0x0000005C	EXTI1_IRQHandler
	6	0x00000058	EXTI0_IRQHandler
	5	0x00000054	RCC_IRQHandler
	4	0x00000050	FLASH_IRQHandler
	3	0x0000004C	RTC_WKUP_IRQHandler
	2	0x00000048	TAMPER_STAMP_IRQHandler
	1	0x00000044	PVD_IRQHandler
	0	0x00000040	WWDG_IRQHandler
	-1	0x0000003C	SysTick_Handler
	-2	0x00000038	PendSV_Handler
	-3	0x00000034	Reserved
	-4	0x00000030	DebugMon_Handler
	-5	0x0000002C	SVC_Handler
	-6	0x00000028	Reserved
	-7	0x00000024	Reserved
	-8	0x00000020	Reserved
	-9	0x0000001C	Reserved
	-10	0x00000018	UsageFault_Handler
	-11	0x00000014	BusFault_Handler
	-12	0x00000010	MemManage_Handler
	-13	0x0000000C	HardFault_Handler
	-14	0x00000008	NMI_Handler
		0x00000004	Reset_Handler
		0x00000000	Top_of_Stack
Interrupt Number		Memory Address	Memory Contents (32 bits)

Program Status Register (PSR)



Looking up IRQs

How do we calculate the address that holds the pointer of the ISR for interrupt n ?

Address of pointer = $64 + 4 \times n$

Example: $\text{EXTI1_IRQn} = 7$

Address of pointer to EXTI1 ISR = $64 + 4 \times 7$
= 92
= **0x5C**

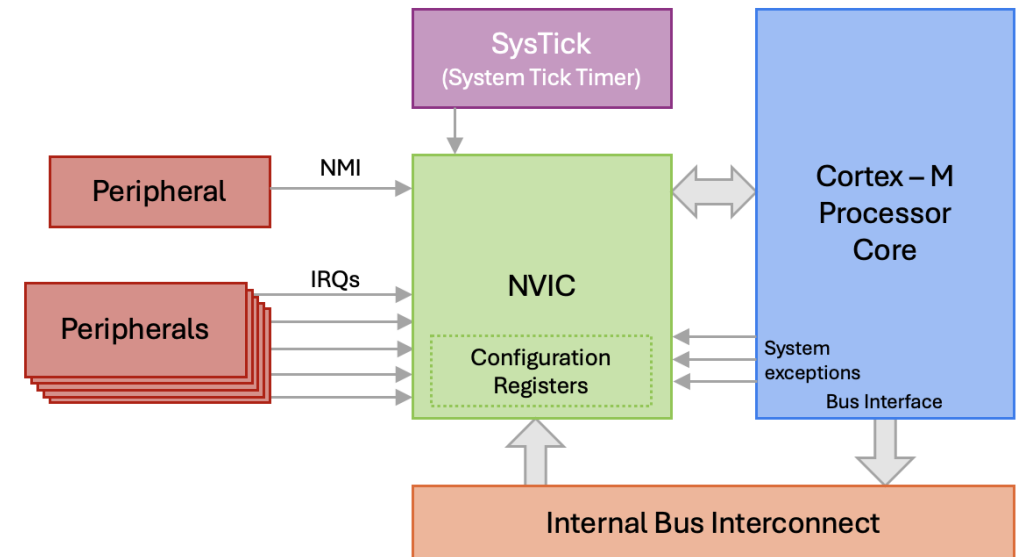
Example: $\text{SysTick_IRQn} = -1$

Address of pointer to EXTI1 ISR = $64 + 4 \times (-1)$
= 60
= **0x3C**

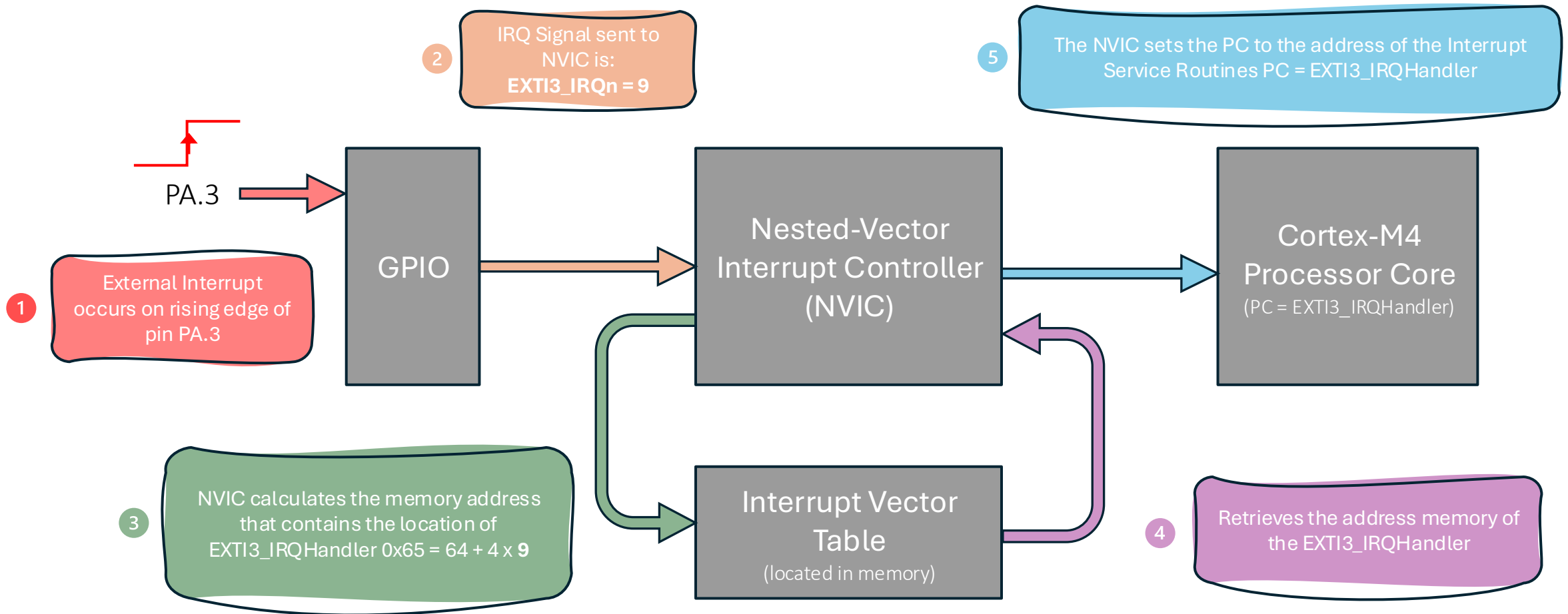
Interrupt Number	Memory Address	Memory Contents (32 bits)
13	0x00000074	DMA1_Channel3_IRQHandler
12	0x00000070	DMA1_Channel2_IRQHandler
11	0x0000006C	DMA1_Channel1_IRQHandler
10	0x00000068	EXTI4_IRQHandler
9	0x00000064	EXTI3_IRQHandler
8	0x00000060	EXTI2_IRQHandler
7	0x0000005C	EXTI1_IRQHandler
6	0x00000058	EXTI0_IRQHandler
5	0x00000054	RCC_IRQHandler
4	0x00000050	FLASH_IRQHandler
3	0x0000004C	RTC_WKUP_IRQHandler
2	0x00000048	TAMPER_STAMP_IRQHandler
1	0x00000044	PVD_IRQHandler
0	0x00000040	WWDG_IRQHandler
-1	0x0000003C	SysTick_Handler
-2	0x00000038	PendSV_Handler
-3	0x00000034	Reserved
-4	0x00000030	DebugMon_Handler
-5	0x0000002C	SVC_Handler
-6	0x00000028	Reserved
-7	0x00000024	Reserved
-8	0x00000020	Reserved
-9	0x0000001C	Reserved
-10	0x00000018	UsageFault_Handler
-11	0x00000014	BusFault_Handler
-12	0x00000010	MemManage_Handler
-13	0x0000000C	HardFault_Handler
-14	0x00000008	NMI_Handler
	0x00000004	Reset_Handler
	0x00000000	Top_of_Stack

Interrupt Signal vs. Interrupt Number

- Each unique interrupt signal is associated with a unique interrupt number
- This association allows the Nested Vectored Interrupt Controller (NVIC) to call the correct Interrupt Service Routine (ISR)
- The location and content of the ISR is defined by the programmer



Quick Overview



Recap: Application Binary Interface (ABI)

Defines:

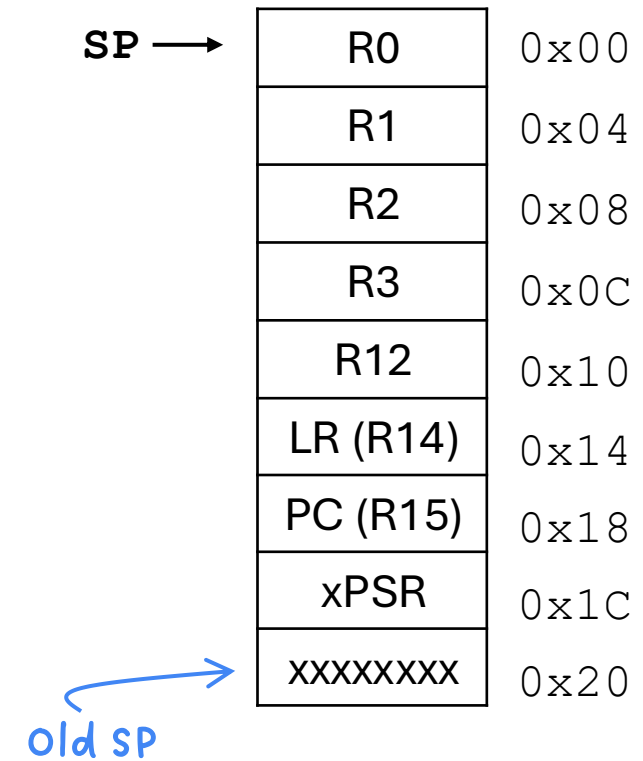
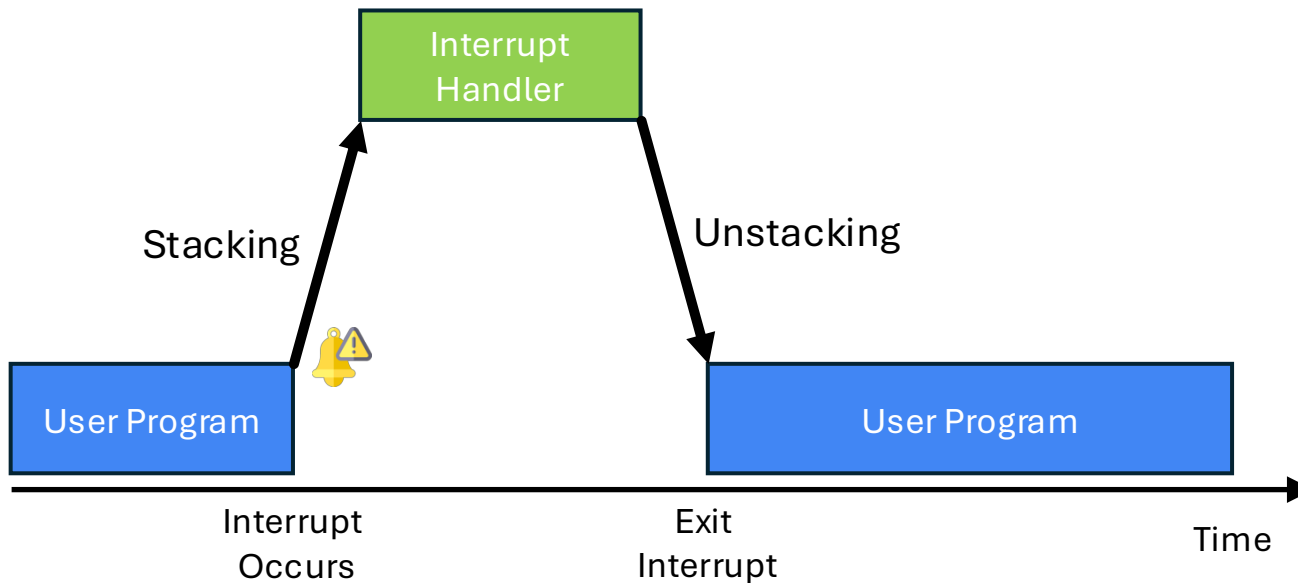
How compiled programs should interface at the binary level...how functions receive arguments and return values

Some Basic Rules:

1. Registers R0, R1, R2, and R3 are used to pass input parameters into a C function
2. Functions must preserve the values of registers R4-R11
3. Return parameter is placed in R0
4. Push and pop an even number of registers to maintain an 8-byte alignment in the stack.

ISR Stacking and Unstacking

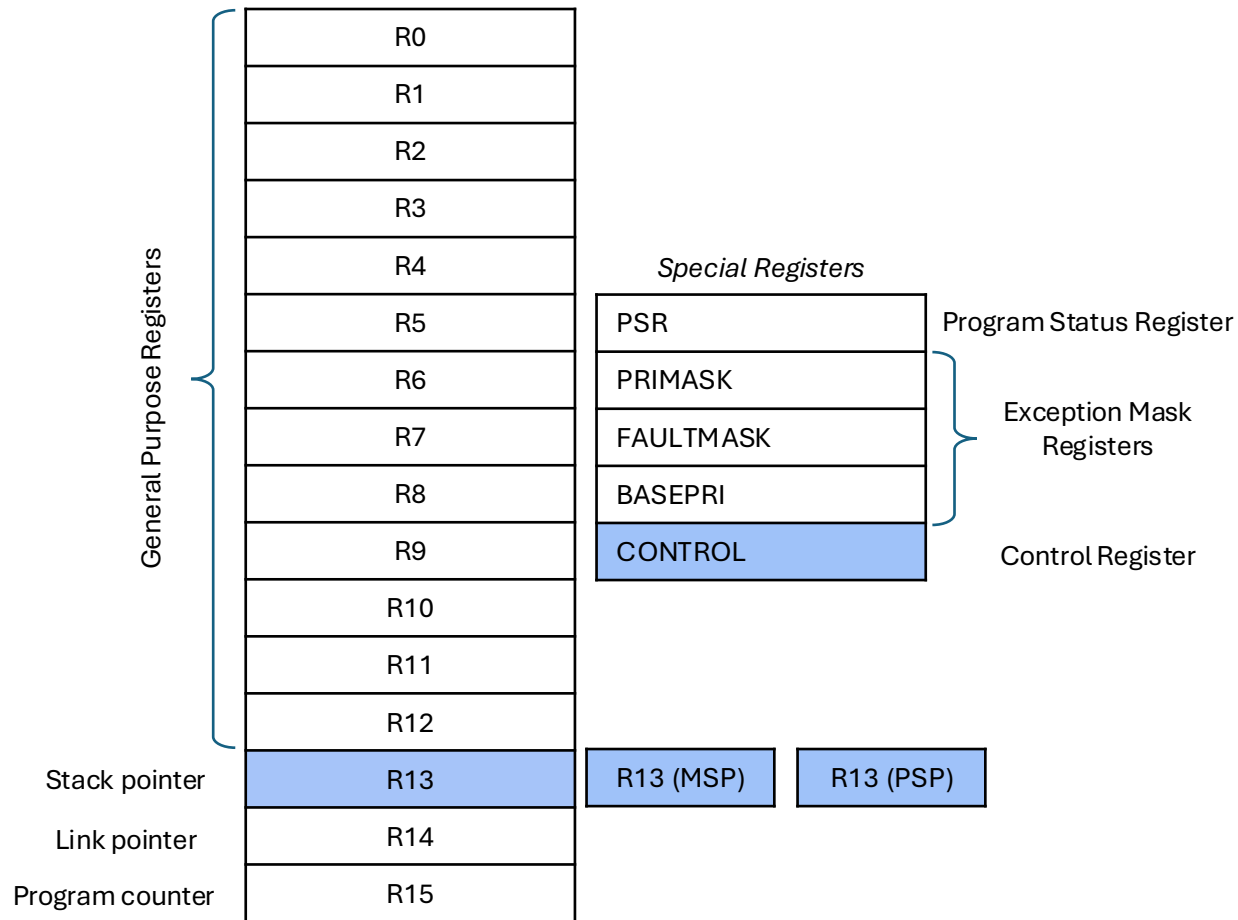
ISR Stacking and Unstacking



Stacking: When an interrupt occurs, the processor automatically pushes the eight registers into the main stack before an interrupt handler starts

Unstacking: The processor automatically pops these eight registers out of the main stack when an interrupt handler exits

Special Register Modes



Main Stack Pointer (MSP)

Used for simple bare-metal applications
In this class we will always use the MSP

Process Stack Pointer (PSP)

Used for OS applications

Which stack to use when exiting an interrupt?

The Link Register (LR) has two usages:

- 1 LR = address of the instruction immediately after BL
- 2 LR indicates whether MSP or PSP is used to restore registers when exiting an interrupt

Link Register (LR)	Return Mode	Return Stack
0xFFFFFFFF1	Handler	SP = MSP
0xFFFFFFFF9	Thread	SP = MSP
0xFFFFFFFDD	Thread	SP = PSP

Example

_main PROC

...

MOV r3, #0

...

ENDP

SysTick_Handler PROC

EXPORT SysTick_Handler

ADD r3, #1

ADD r4, #1

BX lr

ENDP

Address =
0x08000044



R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13 (SP)	0x20000200
R14 (LR)	0x08001000
R15 (PC)	0x08000044

PSR 0x21000000

xxxxxxxx	0x20000200
	0x200001FC
	0x200001F8
	0x200001F4
	0x200001F0
	0x200001EC
	0x200001E8
	0x200001E4
	0x200001E0
	0x200001DC
	0x200001D0
	0x200001C5

Memory

Address = 0x0800001C

Example

_main PROC

...



MOV r3, #0

...

ENDP

SysTick_Handler PROC

EXPORT SysTick_Handler

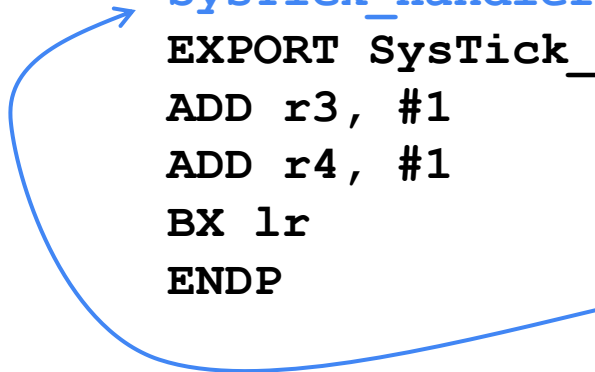
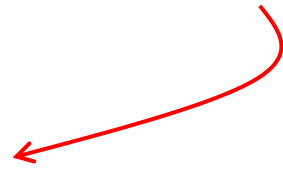
ADD r3, #1

ADD r4, #1

BX lr

ENDP

Address =
0x08000044



Address =
0x0800001C

R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13 (SP)	0x200001E0
R14 (LR)	0xFFFFFFFF9
R15 (PC)	0x0800001C

PSR	0x21000000
-----	------------

	xxxxxxx	0x20000200
PSR	0x21000000	0x200001FC
PC	0x08000044	0x200001F8
LR	0x08001000	0x200001F4
R12	12	0x200001F0
R3	4	0x200001EC
R2	3	0x200001E8
R1	2	0x200001E4
R0	1	0x200001E0
		0x200001DC
		0x200001D0
		0x200001C5

Memory

Example

_main PROC

...



MOV r3, #0

...

ENDP

SysTick_Handler PROC

EXPORT SysTick_Handler

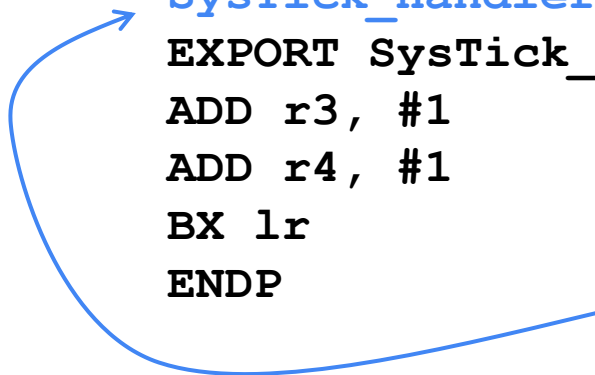
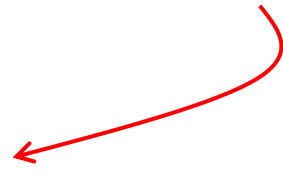
ADD r3, #1

ADD r4, #1

BX lr

ENDP

Address =
0x08000044



Address =
0x0800001C

R0	0
R1	1
R2	2
R3	4
R4	5
R12	12
R13 (SP)	0x200001E0
R14 (LR)	0xFFFFFFFF9
R15 (PC)	0x0800001C

PSR	0x21000000
-----	------------

	xxxxxxx	0x20000200
PSR	0x21000000	0x200001FC
PC	0x08000044	0x200001F8
LR	0x08001000	0x200001F4
R12	12	0x200001F0
R3	4	0x200001EC
R2	3	0x200001E8
R1	2	0x200001E4
R0	1	0x200001E0
		0x200001DC
		0x200001D0
		0x200001C5

Memory

Example

```
_main PROC
```

```
...
```

```
MOV r3, #0
```

```
...
```

```
ENDP
```

```
SysTick_Handler PROC
```

```
EXPORT SysTick_Handler
```

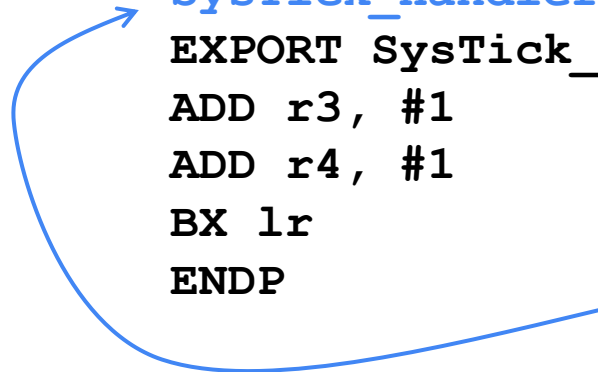
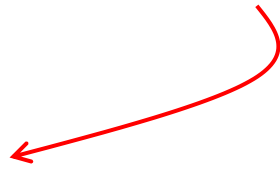
```
ADD r3, #1
```

```
ADD r4, #1
```

```
BX lr
```

```
ENDP
```

Address =
0x08000044



Address =
0x0800001C

R0	0
R1	1
R2	2
R3	3
R4	5
R12	12
R13 (SP)	0x20000200
R14 (LR)	0x08001000
R15 (PC)	0x08000044

PSR	0x21000000
-----	------------

	PSR	xxxxxxxx	0x20000200
			0x200001FC
	PC		0x200001F8
	LR		0x200001F4
	R12		0x200001F0
	R3		0x200001EC
	R2		0x200001E8
	R1		0x200001E4
	R0		0x200001E0
			0x200001DC
			0x200001D0
			0x200001C5

Memory

Open Questions

How do we set up and control interrupts?

What if some interrupts are more important than others?

What if the processor gets an interrupt while it is handling another interrupt?

What if we are in a “critical section of code?”

Interrupt Control Registers

Recap: Interrupt Number

ARM Cortex – M Supports up to 256 interrupts.

- The first **16 are system exceptions**.
 - CMSIS defines their interrupt numbers as negative
 - Defined by ARM core
- The remaining **240 are peripheral interrupts**.
 - Peripheral interrupt number starts with 0
 - Defined by chip manufacturers
- **STM32L4R5 has 96 peripheral interrupts**

Interrupt Control Registers

- Interrupt Controller Type Register (ICTR), Read/Write
- Interrupt Set-Enable Register (ISER), Read/Write
- Interrupt Clear-Enable Register (ICER), Read/Write
- Interrupt Set-Pending Register (ISPR), Read/Write
- Interrupt Clear-Pending Register (ICPR), Read/Write
- Interrupt Active Bit Register (IABR), Read only
- Software Trigger Interrupt Register (STIR), Write only
- Interrupt Priority Register (IPR), Read/Write
- Application Interrupt and Reset Control (AIRC)

CMSIS Access NVIC Functions

Enable an interrupt	<code>void NVIC_EnableIRQ(IRQn_Type IRQn);</code>
Disable an interrupt	<code>void NVIC_DisableIRQ(IRQn_Type IRQn);</code>
Set an interrupt line as pending	<code>void NVIC_SetPendingIRQ(IRQn_Type IRQn);</code>
Clear the pending flag	<code>void NVIC_ClearPendingIRQ(IRQn_Type IRQn);</code>
Check if IRQ is pending	<code>uint32_t NVIC_GetPendingIRQ(IRQn_Type IRQn);</code>
Check if IRQ is active	<code>uint32_t NVIC_GetActive(IRQn_Type IRQn);</code>
Set the interrupt priority	<code>void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority);</code>
Get the interrupt priority	<code>uint32_t NVIC_GetPriority(IRQn_Type IRQn);</code>

Register Layout Example: ISER

Interrupt Set Enable Register 0 (ISER0)

Register Index

Enable Bit

Interrupt Number

Interrupt Set Enable Register 0 (ISER0)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
FAB_IRQ	Reserved	Reserved	Reserved	Reserved	Reserved	Reserved	Reserved	BM_ERROR_IRQ	PLLCLKLOST_IRQ	PLLCLK_IRQ	TIMER_2_IRQ	TIMER_1_IRQ	I2C_1_SMBUS_IRQ	I2C_1_SMBALERT_IRQ	I2C_1_IRQ	I2C_0_SMBUS_IRQ	I2C_0_SMBALERT_IRQ	I2C_0_IRQ	SPI_1_IRQ	SPI_0_IRQ	UART_1_IRQ	UART_0_IRQ	DMA_IRQ	ENVM_1_IRQ	ENVM_0_IRQ	M3_IAP_IRQ	EMAC_IRQ	PU_N_IRQ	RTCMATCHEVENT_IRQ	BROWNOUT_3V_IRQ	BROWNOUT_5V_IRQ	WDOGWAKEUP_IRQ

Interrupt Set Enable Register 1 (ISER1)

Interrupt Set Enable Register 1 (ISER1)																																
Register Index	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Enable Bit	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Interrupt Number	63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48	47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
	GPIO_31_IRQ	GPIO_30_IRQ	GPIO_29_IRQ	GPIO_28_IRQ	GPIO_27_IRQ	GPIO_26_IRQ	GPIO_25_IRQ	GPIO_24_IRQ	GPIO_23_IRQ	GPIO_22_IRQ	GPIO_21_IRQ	GPIO_20_IRQ	GPIO_19_IRQ	GPIO_18_IRQ	GPIO_17_IRQ	GPIO_16_IRQ	GPIO_15_IRQ	GPIO_14_IRQ	GPIO_13_IRQ	GPIO_12_IRQ	GPIO_11_IRQ	GPIO_10_IRQ	GPIO_9_IRQ	GPIO_8_IRQ	GPIO_7_IRQ	GPIO_6_IRQ	GPIO_5_IRQ	GPIO_4_IRQ	GPIO_3_IRQ	GPIO_2_IRQ	GPIO_1_IRQ	GPIO_0_IRQ

Register Layout Example: ISER

Interrupt Set-Enable Register

ISER0	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ISER1	63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48	47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
ISER2	95	94	93	92	91	90	89	87	86	85	84	83	82	81	80	79	78	77	76	75	74	73	72	71	70	69	68	67	66	65	64	63

STM32L4R5 only has 96 interrupts + 16 ARM CMSIS

ISER and ICER

Core peripherals

PM0214

4.3.2 Interrupt set-enable register x (NVIC_ISERx)

Address offset: $0x100 + 0x04 * x$, ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

Required privilege: Privileged

NVIC_ISER0 bits 0 to 31 are for interrupt 0 to 31, respectively

NVIC_ISER1 bits 0 to 31 are for interrupt 32 to 63, respectively

....

NVIC_ISER6 bits 0 to 31 are for interrupt 192 to 223, respectively

NVIC_ISER7 bits 0 to 15 are for interrupt 224 to 239, respectively

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SETENA[31:16]															
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SETENA[15:0]															
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs

Bits 31:0 **SETENA**: Interrupt set-enable bits.

Write:

- 0: No effect
- 1: Enable interrupt

Read:

- 0: Interrupt disabled
- 1: Interrupt enabled.

If a pending interrupt is enabled, the NVIC activates the interrupt based on its priority. If an interrupt is not enabled, asserting its interrupt signal changes the interrupt state to pending, but the NVIC never activates the interrupt, regardless of its priority.

Bits 16 to 31 of the NVIC_ISER7 register are reserved.

Note: *The number of interrupts is product-dependent. Refer to reference manual/datasheet of relevant STM32 product for related information.*

PM0214

Core peripherals

4.3.3 Interrupt clear-enable register x (NVIC_ICERx)

Address offset: $0x180 + 0x04 * x$, ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

Required privilege: Privileged

NVIC_ICER0 bits 0 to 31 are for interrupt 0 to 31, respectively

NVIC_ICER1 bits 0 to 31 are for interrupt 32 to 63, respectively

....

NVIC_ICER6 bits 0 to 31 are for interrupt 192 to 223, respectively

NVIC_ICER7 bits 0 to 15 are for interrupt 224 to 239, respectively

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CLRENA[31:16]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CLRENA[15:0]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:0 **CLRENA**: Interrupt clear-enable bits.

Write:

- 0: No effect
- 1: Disable interrupt

Read:

- 0: Interrupt disabled
- 1: Interrupt enabled.

Bits 16 to 31 of the NVIC_ICER7 register are reserved.

Note: *The number of interrupts is product-dependent. Refer to reference manual/datasheet of relevant STM32 product for related information.*

ISPR and ICPR (Pending and Clear Pending)

Core peripherals

PM0214

4.3.4 Interrupt set-pending register x (NVIC_ISPRx)

Address offset: $0x200 + 0x04 * x$, ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

Required privilege: Privileged

NVIC_ISPR0 bits 0 to 31 are for interrupt 0 to 31, respectively

NVIC_ISPR1 bits 0 to 31 are for interrupt 32 to 63, respectively

....

NVIC_ISPR6 bits 0 to 31 are for interrupt 192 to 223, respectively

NVIC_ISPR7 bits 0 to 15 are for interrupt 224 to 239, respectively

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SETPEND[31:16]															
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SETPEND[15:0]															
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs

Bits 31:0 **SETPEND**: Interrupt set-pending bits

Write:

0: No effect

1: Changes interrupt state to pending

Read:

0: Interrupt is not pending

1: Interrupt is pending

Writing 1 to the ISPR bit corresponding to an interrupt that is pending has no effect.

Writing 1 to the ISPR bit corresponding to a disabled interrupt sets the state of that interrupt to pending.

Bits 16 to 31 of the NVIC_ISPR7 register are reserved.

PM0214

Core peripherals

4.3.5 Interrupt clear-pending register x (NVIC_ICPRx)

Address offset: $0x280 + 0x04 * x$, ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

Required privilege: Privileged

NVIC_ICPR0 bits 0 to 31 are for interrupt 0 to 31, respectively

NVIC_ICPR1 bits 0 to 31 are for interrupt 32 to 63, respectively

....

NVIC_ICPR6 bits 0 to 31 are for interrupt 192 to 223, respectively

NVIC_ICPR7 bits 0 to 15 are for interrupt 224 to 239, respectively

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CLRPEND[31:16]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CLRPEND[15:0]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:0 **CLRPEND**: Interrupt clear-pending bits

Write:

0: No effect

1: Removes the pending state of an interrupt

Read:

0: Interrupt is not pending

1: Interrupt is pending

Writing 1 to an ICPR bit does not affect the active state of the corresponding interrupt.

Bits 16 to 31 of the NVIC_ICPR7 register are reserved.

Interrupt Active Status Register (IABR)

Indicates which interrupt is active (Read only)

Core peripherals

PM0214

4.3.6 Interrupt active bit register x (NVIC_IABRx)

Address offset: $0x300 + 0x04 * x$, ($x = 0$ to 7)

Reset value: 0x0000 0000

Required privilege: Privileged

NVIC_IABR0 bits 0 to 31 are for interrupt 0 to 31, respectively

NVIC_IABR1 bits 0 to 31 are for interrupt 32 to 63, respectively

....

NVIC_IABR6 bits 0 to 31 are for interrupt 192 to 223, respectively

NVIC_IABR7 bits 0 to 15 are for interrupt 224 to 239, respectively

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ACTIVE[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ACTIVE[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **ACTIVE**: Interrupt active flags

0: Interrupt not active

1: Interrupt active

A bit reads as 1 if the status of the corresponding interrupt is active or active and pending.

Bits 16 to 31 of the NVIC_IABR7 register are reserved.

Recap: Interrupt Registers

- Enable and disable registers
- Determine which interrupts are pending and clear them
- Check to see which interrupts are active

Basic Interrupt Flow Control

Pending Interrupt - Simple Case

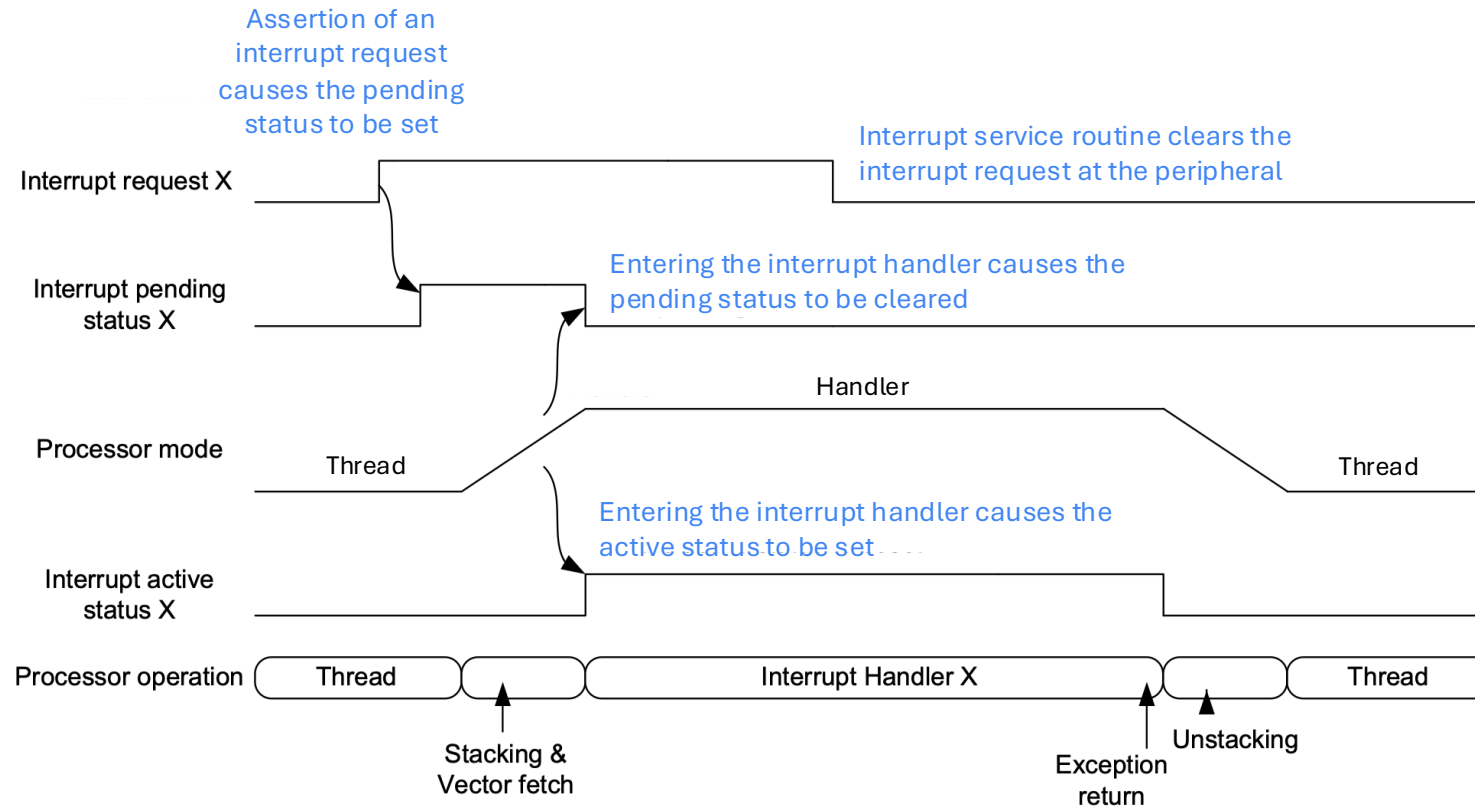
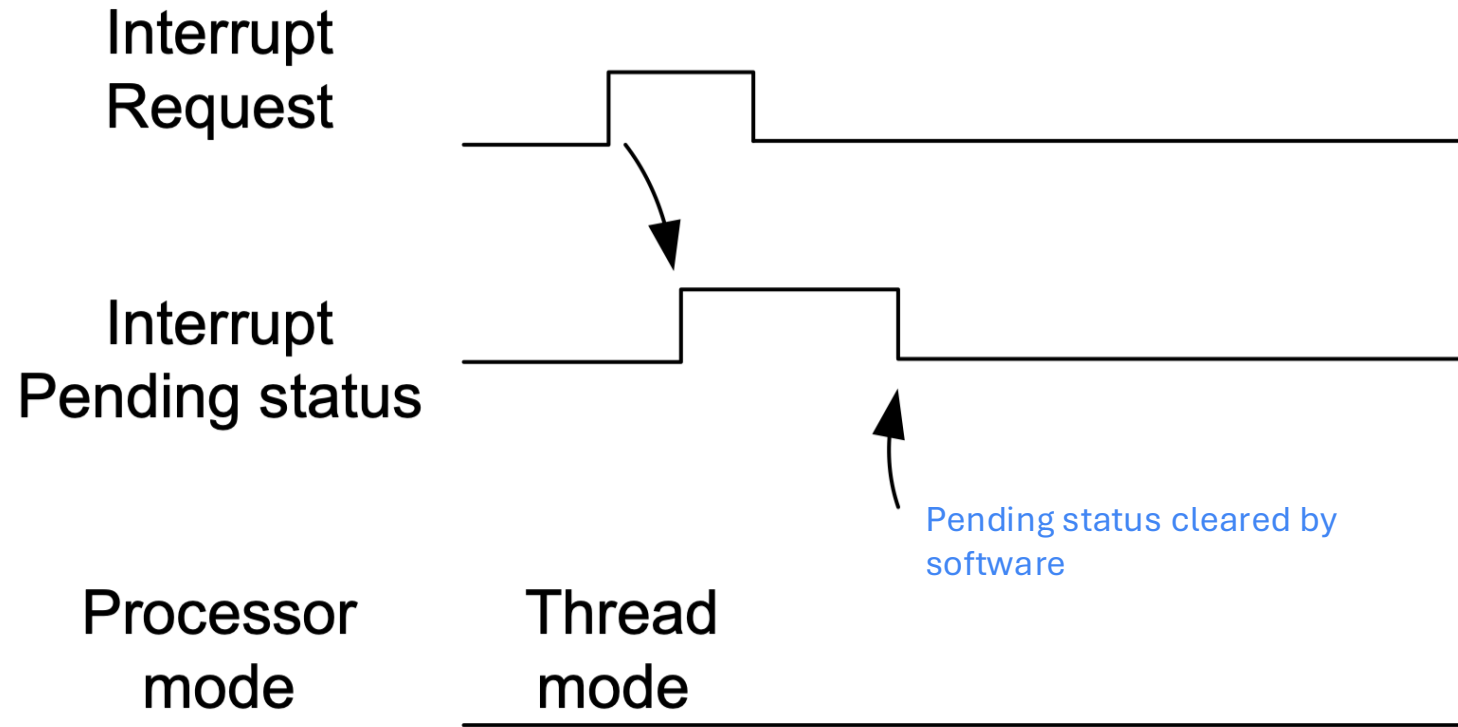


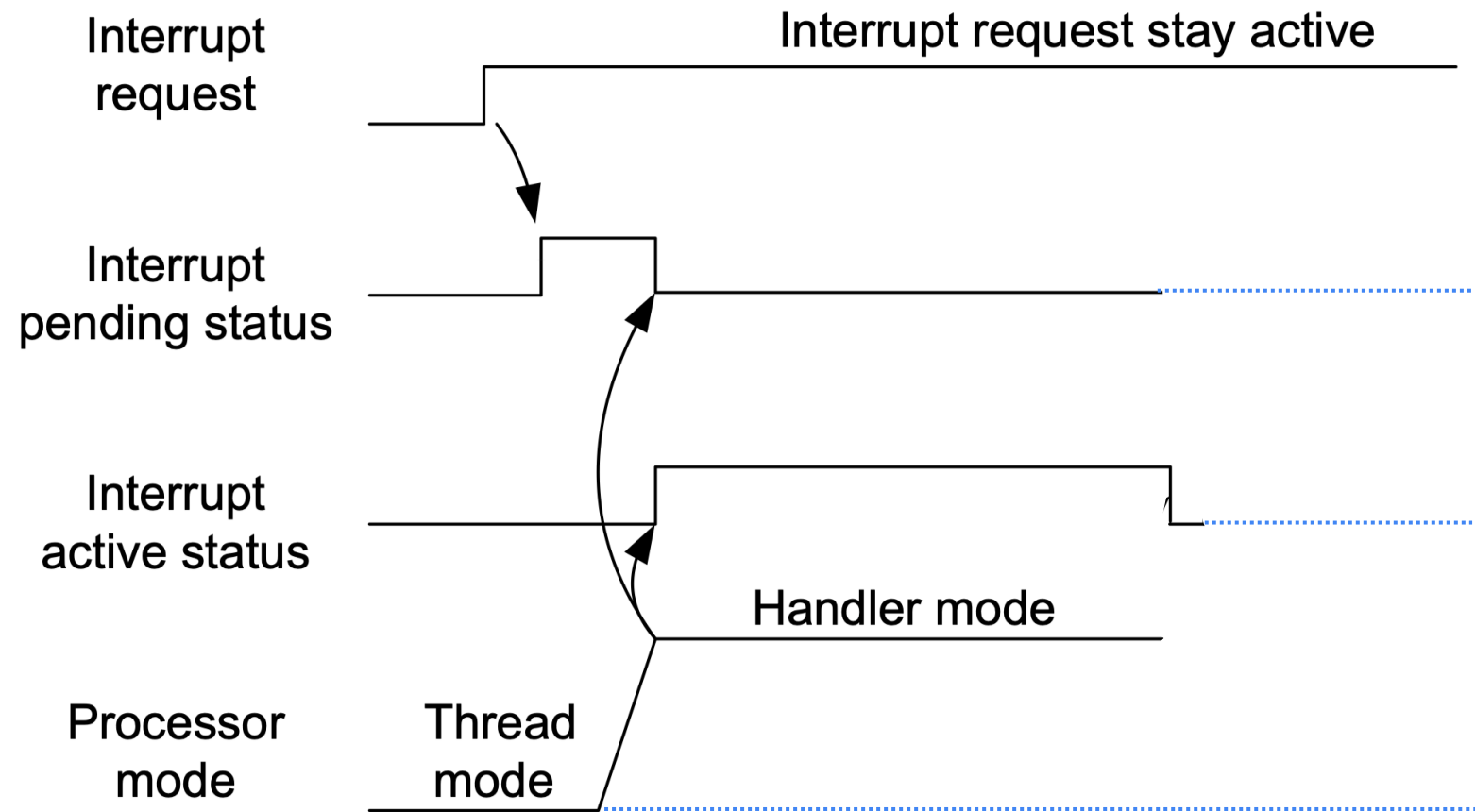
Photo Credit: The Definitive Guide to ARM CORTEX – M3 and CORTEX – M4 Processors

Pending Cleared Before Serving Interrupt



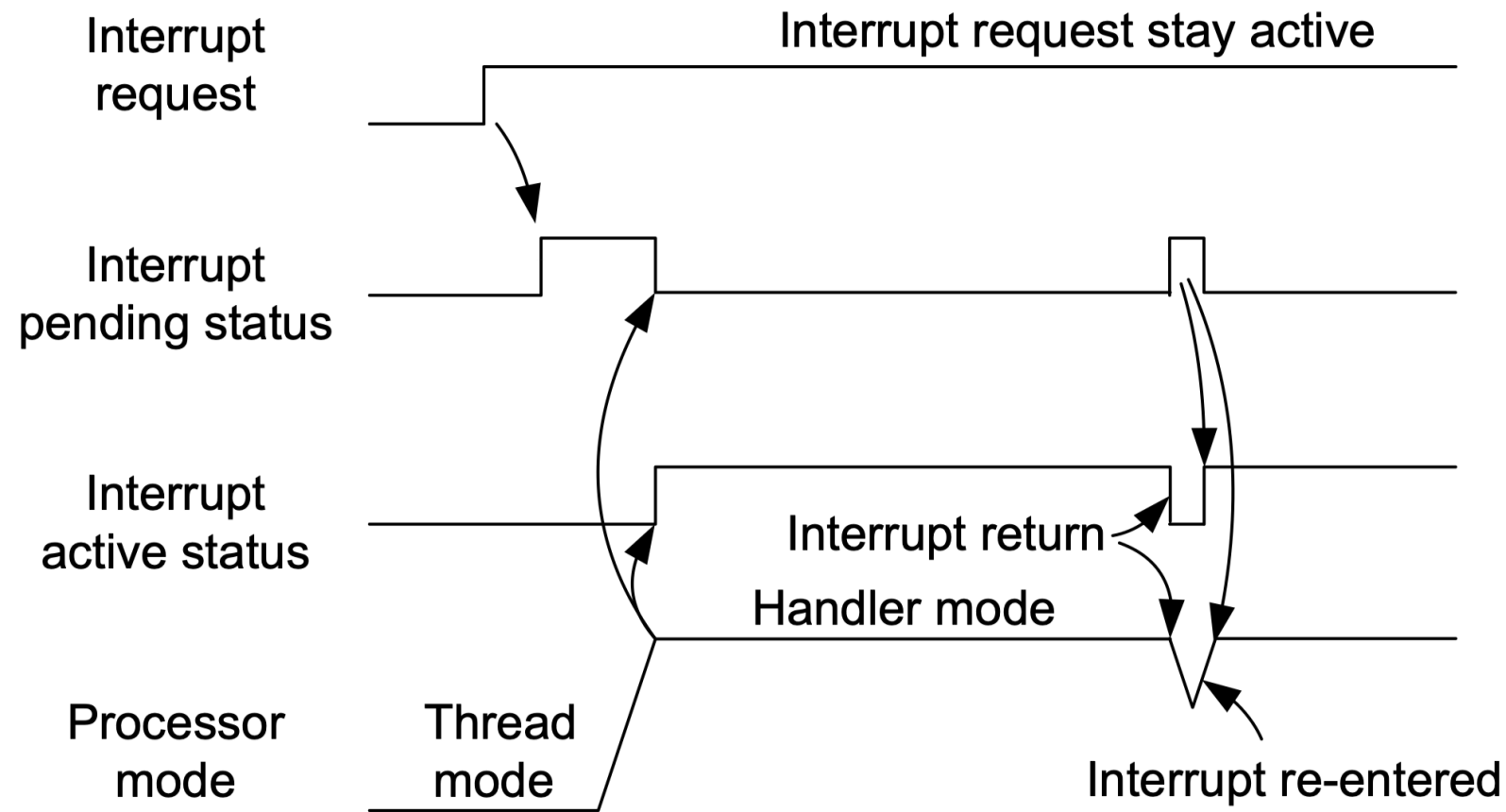
Group Exercise

What happens if the IRQ is *not cleared*?



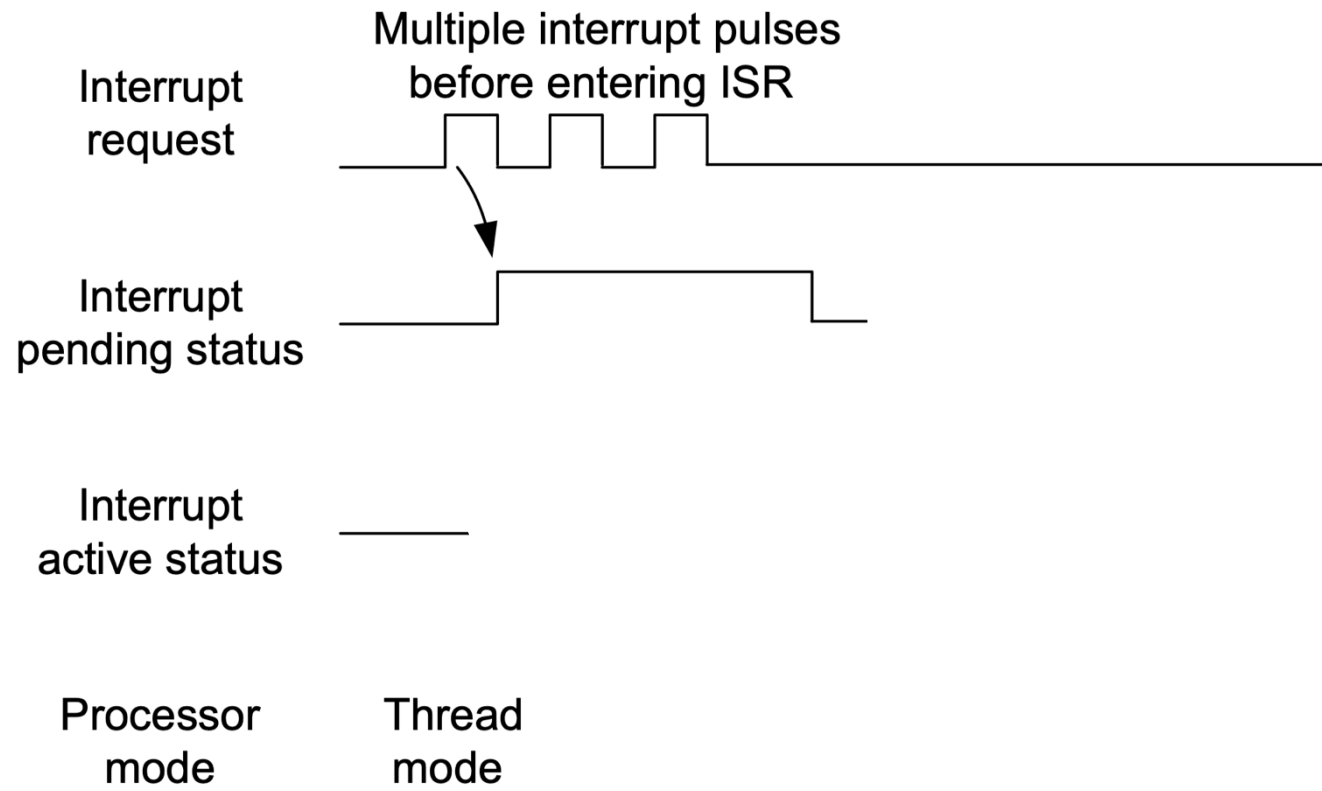
Group Exercise

What happens if the IRQ is *not cleared*?



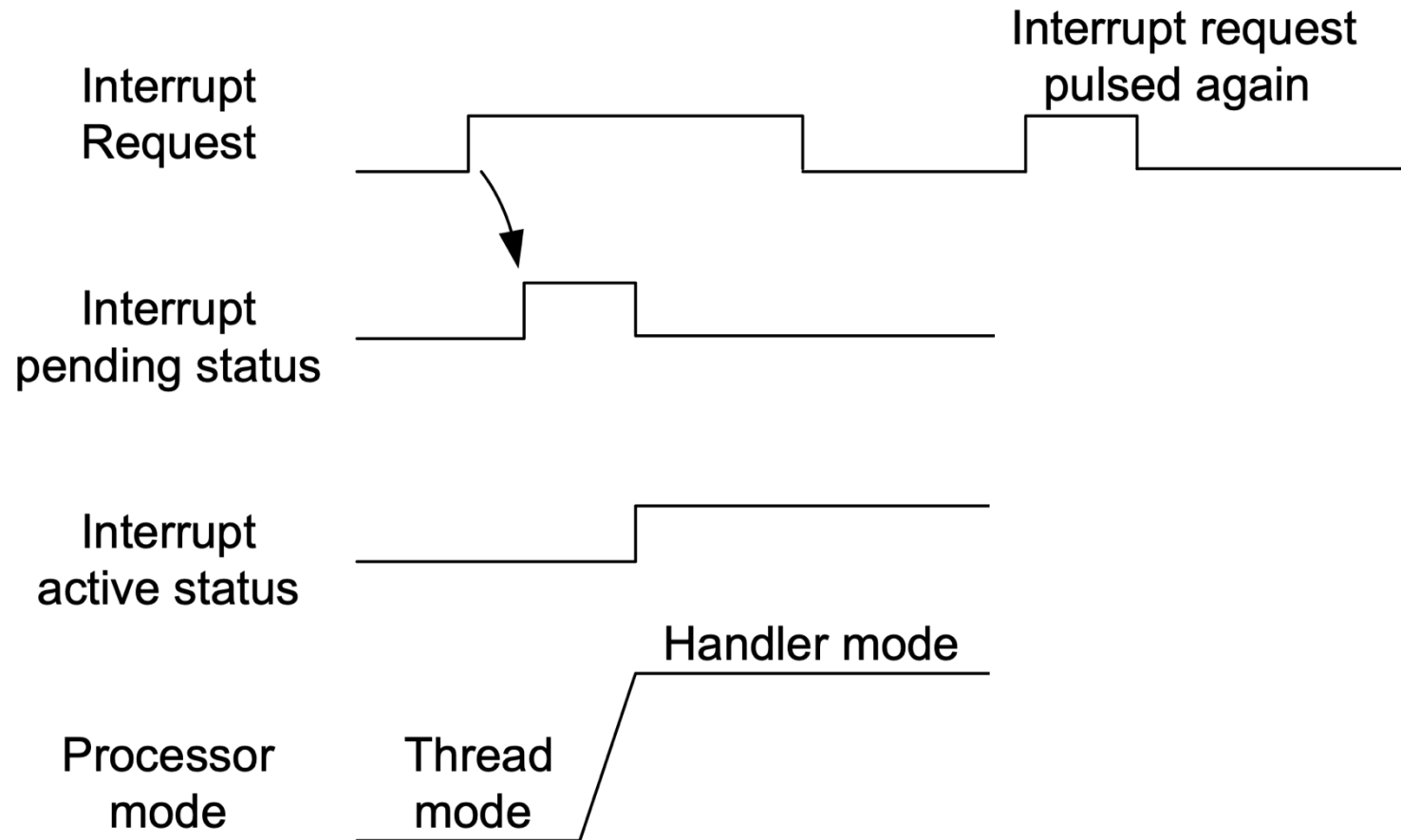
Group Exercise

What happens if the interrupt request signal is pulsed several times *before* the processor starts processing?



Group Exercise

What happens if an *interrupt request occurs again* during the execution of an ISR?



Interrupt Priorities

Interrupt Priority

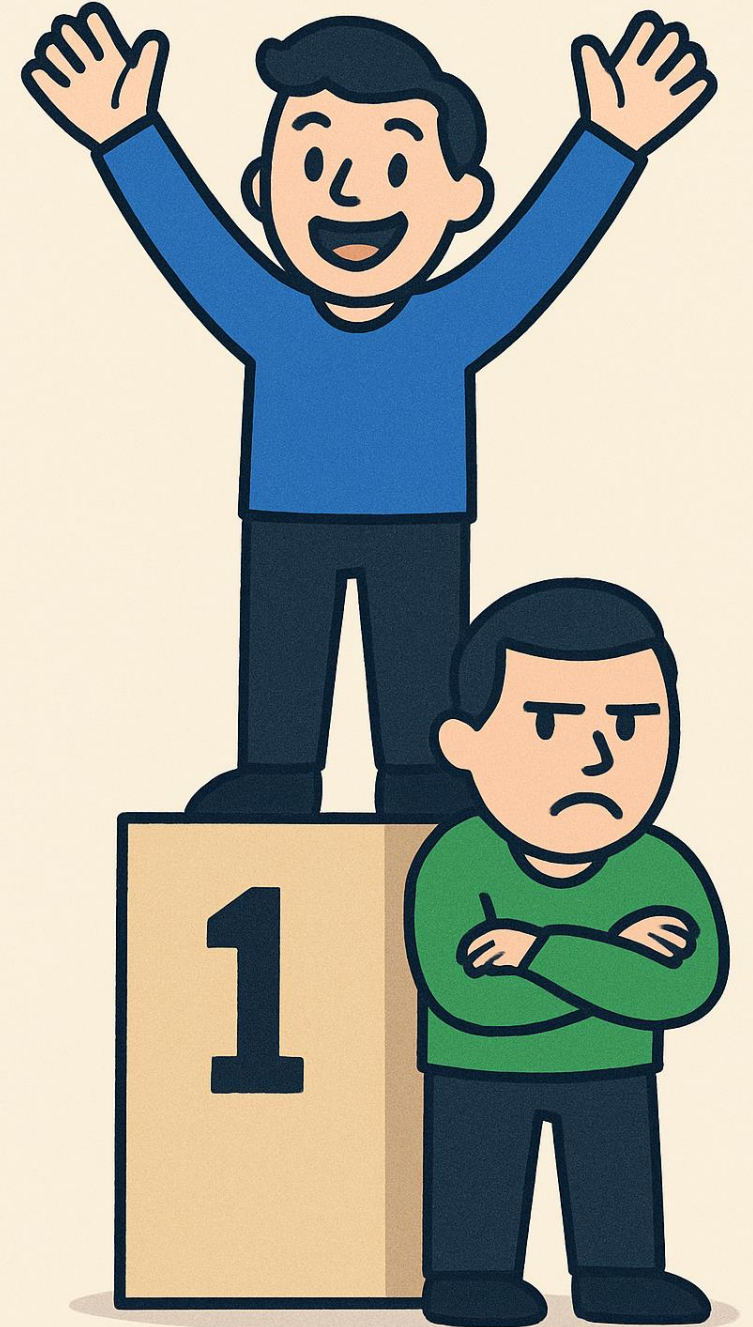
- **Determines the order of service** when two or more requests are made simultaneously
- Allows a **higher priority request to suspend a lower priority** request currently being processed
- Typically, if two requests have the same priority, we do not allow them to interrupt each other
- NVIC assigns a priority level to each interrupt trigger

Address	Name	Type	Description
0xE000E400	PRI_0	R/W	Priority level external interrupt #0
0xE000E401	PRI_1	R/W	Priority level external interrupt #1
...	-	-	-
0xE000E41F	PRI_31	R/W	Priority level external interrupt #31
...	-	-	-

Interrupt Priority

Lower priority means higher urgency

If Interrupt A has a priority of 5 and Interrupt B has a priority of 1, then ***B*** has a higher importance than A



Interrupt Priority

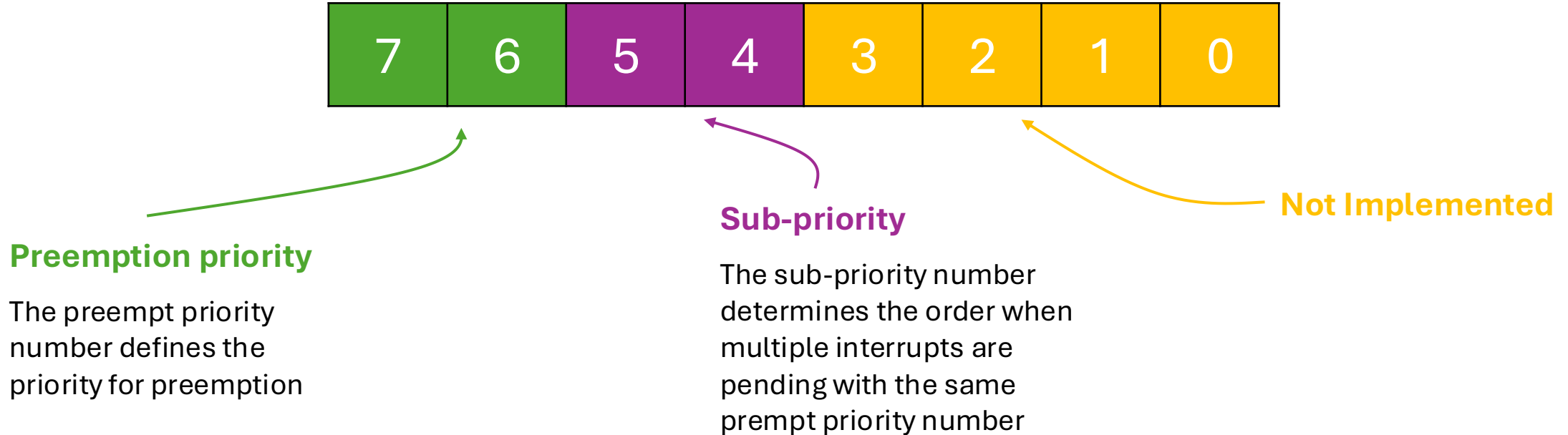
Fixed Priority for Reset, Hard Fault, and NMI

Exception	IRQn	Priority
Reset	N/A	-3 (highest)
Non-maskable Interrupt	-14	-2 (2 nd highest)
Hard Fault	-13	-1 (3 rd highest)

Interrupt Priority

Interrupt priority is configured by the **Interrupt Priority Register (ISR)**

Each priority consists of two fields, including **preempt priority number** and **subpriority number**



What is interrupt preemption?

If the processor is handling one interrupt **and a higher-priority interrupt occurs**, it can pause that work and immediately switch to handling the more important interrupt.

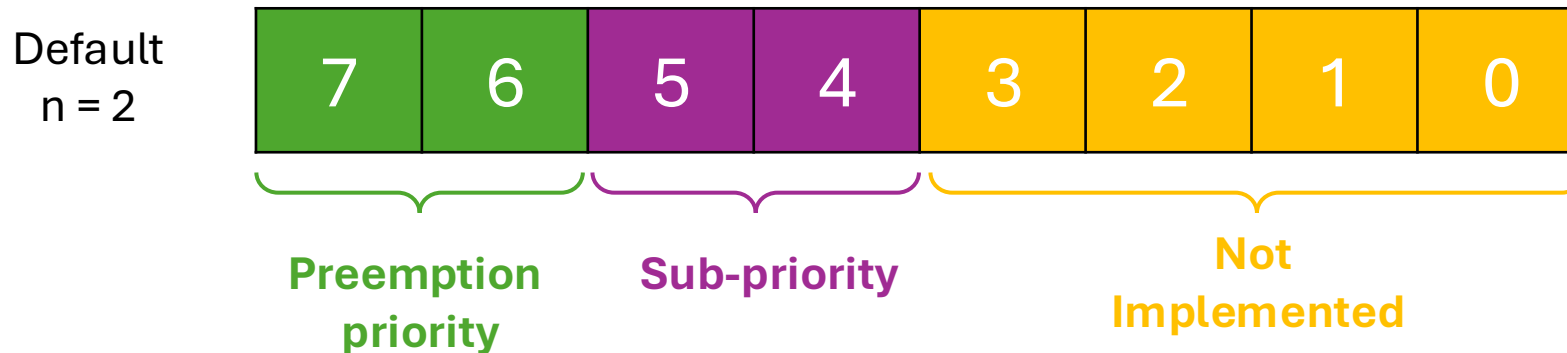


Preemption and Sub-priority Configuration

You can change the allocation of the register bits for preemption and sub-priority

```
NVIC_SetPriorityGrouping(uint32_t PriorityGroup)
```

n	# of preemption bits	# of sub-priority bits
0	0	4
1	1	3
2 (default)	2	2
3	3	1
4	4	0



Masking Priority Register

Base Priority Register (BASEPRI)

Disable all interrupts with less urgency

- For critical code, only certain interrupts are allowed
- **BASEPRI**: Disable all interrupts of specific priority level or higher priority level

```
// Disable interrupts with priority value same or higher
__set_BASEPRI( 5 << 4)

// Critical code
...

// Remove BASEPRI masking
__set_BASEPRI(0);
```

Exception-masking Registers

PRIMASK: Used to disable all exceptions except non-maskable interrupt (NMI) and hard fault

Write 1 to PRIMASK to disable all interrupts except NMI	<pre>MOV R0, #1 MSR PRIMASK, R0</pre>
---	---------------------------------------

Write 0 to PRIMASK to enable all interrupts	<pre>MOV R0, #0 MSR PRIMASK, R0</pre>
--	---------------------------------------

FAULTMASK: Similar to PRIMASK, but change the current priority level to -1, so that even hard fault handler is blocked

BASEPRI: Disable interrupts only with priority lower than the specified level

Disable all exceptions with a priority level higher than 0x60	<pre>MOV R0, #0x60 MSR BASEPRI, R0</pre>
---	--

Example: Single Interrupt

Program

Program
Stack

IRQ
Priority Table

0	8
1	10
1	5
3	3
4	6

Base Priority

256

Pending

0	0	0	0	0
---	---	---	---	---

Enabled

1	1	1	1	1
---	---	---	---	---

Active

0	0	0	0	0
---	---	---	---	---

Example: Single Interrupt



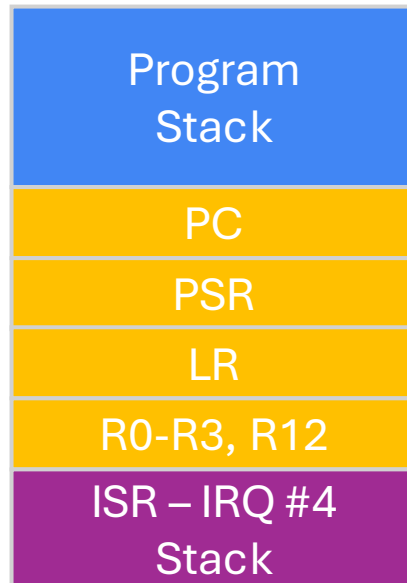
IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6
Base Priority	
256	

Pending				
4				0
1	0	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
0	0	0	0	0

Example: Single Interrupt



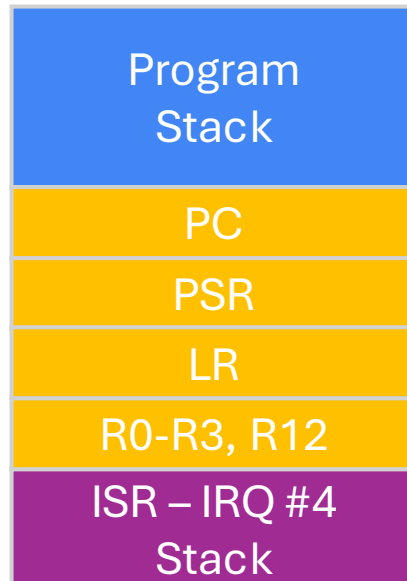
IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6
Base Priority	
	6

Pending				
4				0
0	0	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
1	0	0	0	0

Example: Single Interrupt



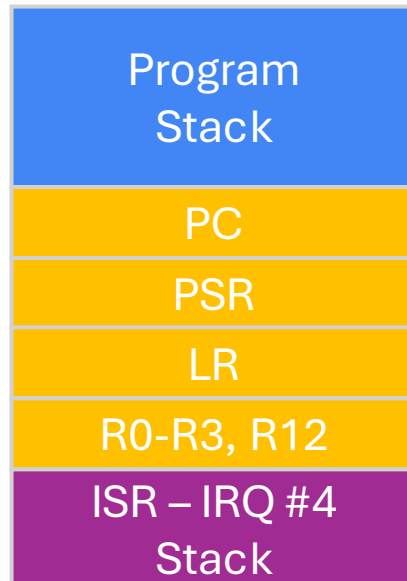
IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6
Base Priority	
	6

Pending				
4				0
0	0	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
1	0	0	0	0

Example: Single Interrupt



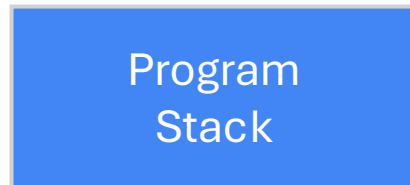
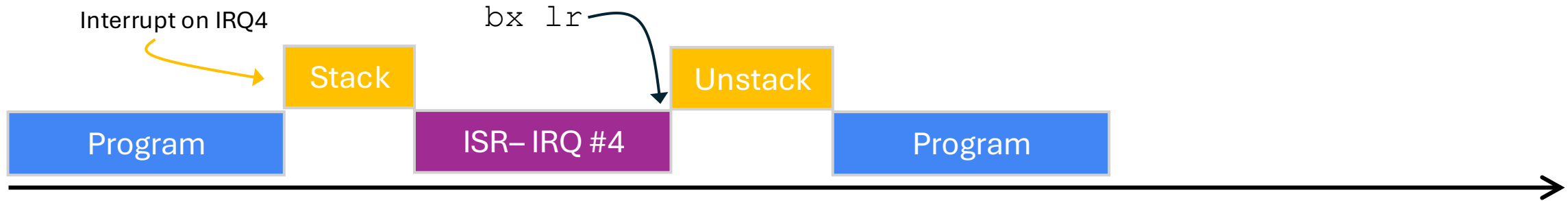
IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6
Base Priority	
	6

Pending				
4				0
0	0	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
0	0	0	0	0

Example: Single Interrupt



IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

Base Priority
256

Pending				
4				0
0	0	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
0	0	0	0	0

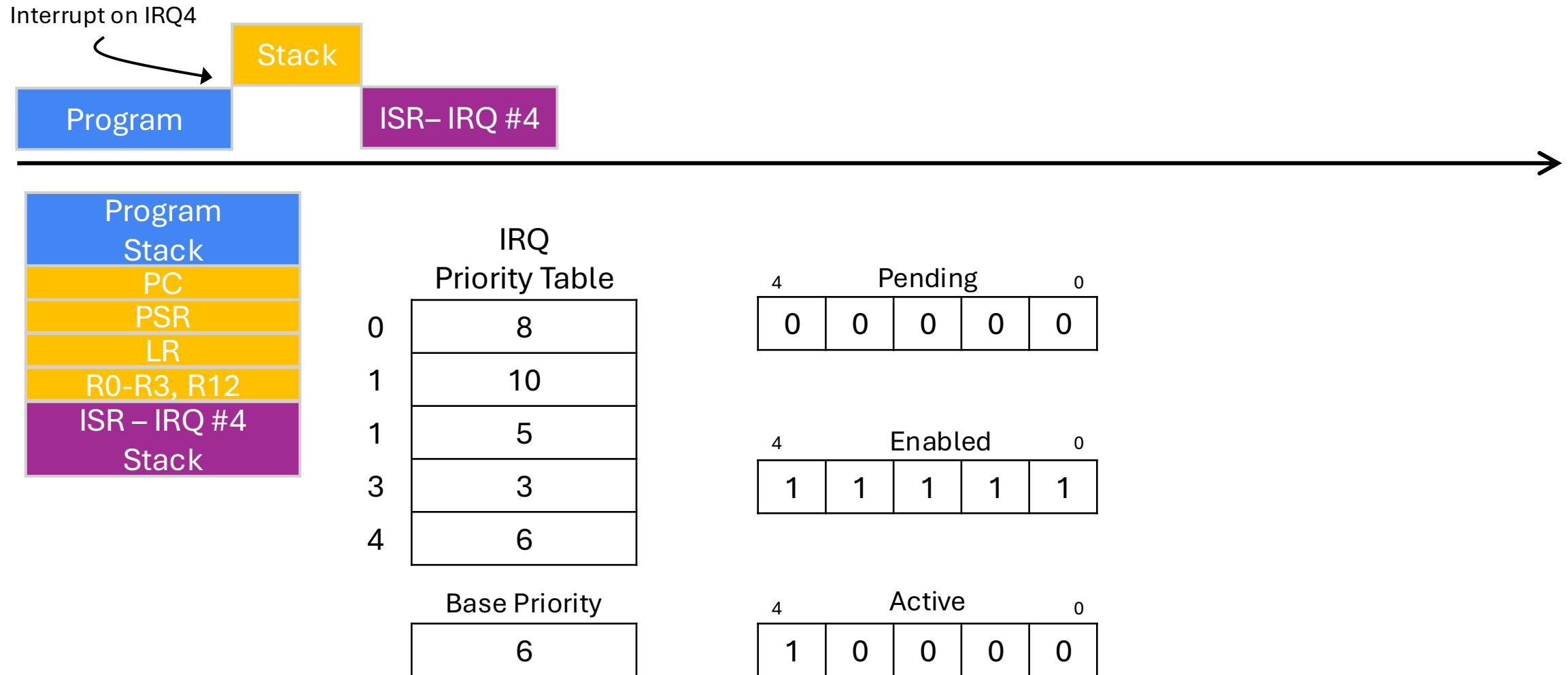
Nested Interrupts / Preemption

Interrupt occurs with **higher priority** while CPU is handling another interrupt of **lower importance** (i.e. incoming request has a lower priority number)

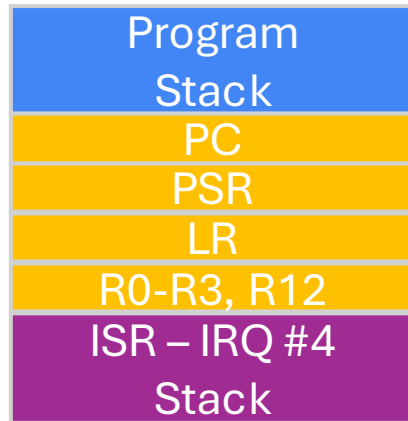
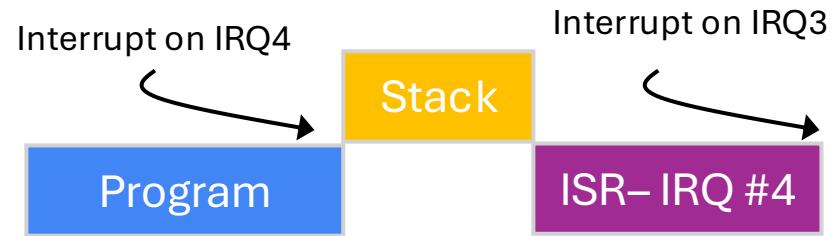
New interrupt will interrupt the current ISR

Will push a new ISR stack on the stack

Example: Nested Interrupt



Example: Nested Interrupt



IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

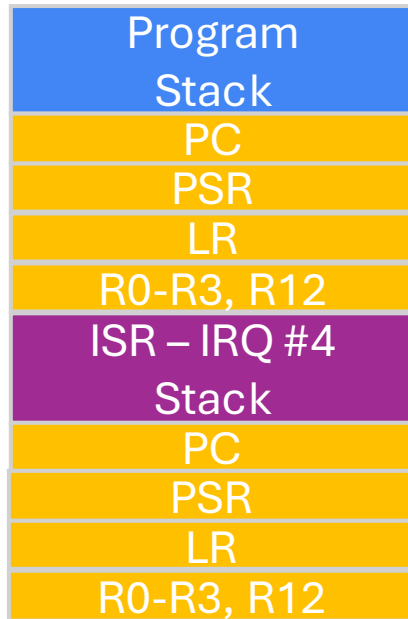
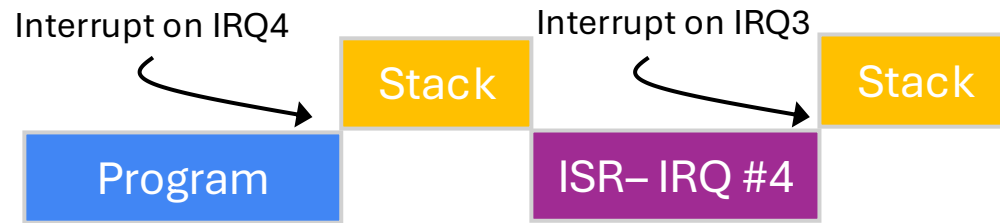
Base Priority	
	6

Pending				
4				0
0	0	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
1	0	0	0	0

Example: Nested Interrupt



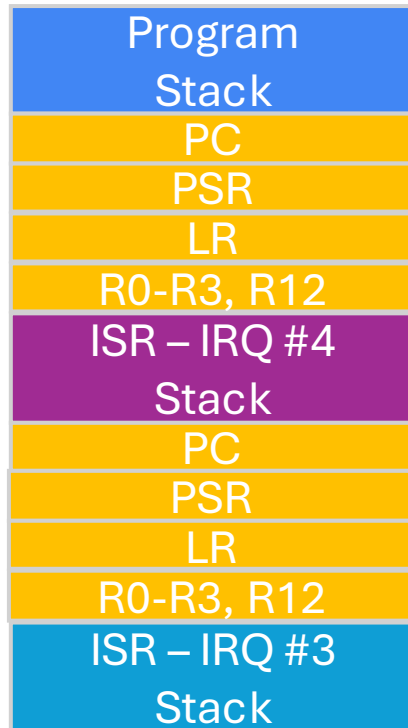
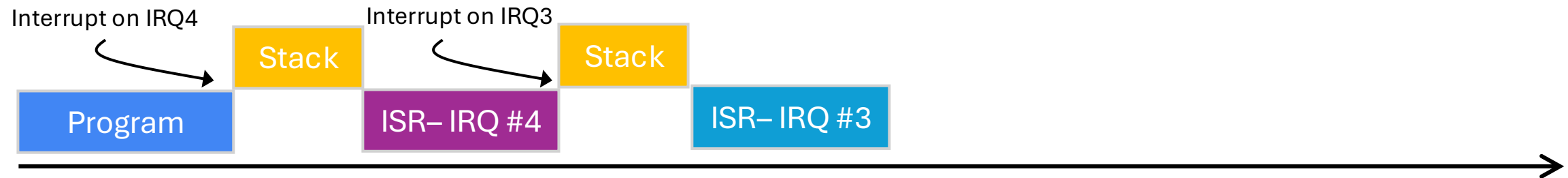
IRQ	
Priority Table	
0	8
1	10
1	5
3	3
4	6
Base Priority	
6	

Pending				
4				0
0	1	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
1	0	0	0	0

Example: Nested Interrupt



IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

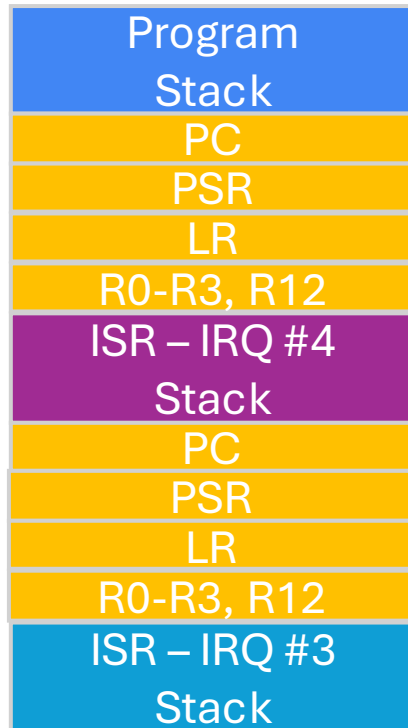
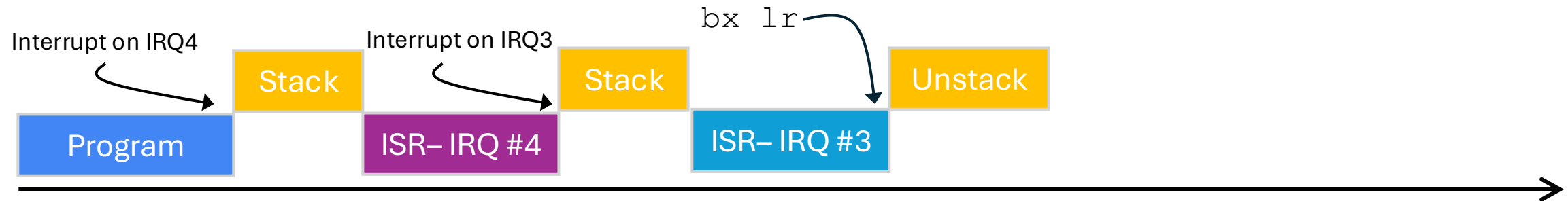
Base Priority	
	3

Pending				
4				0
0	0	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
1	1	0	0	0

Example: Nested Interrupt



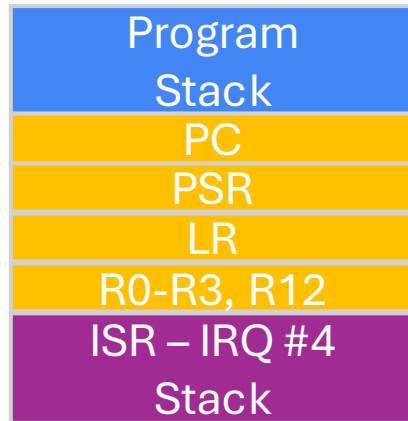
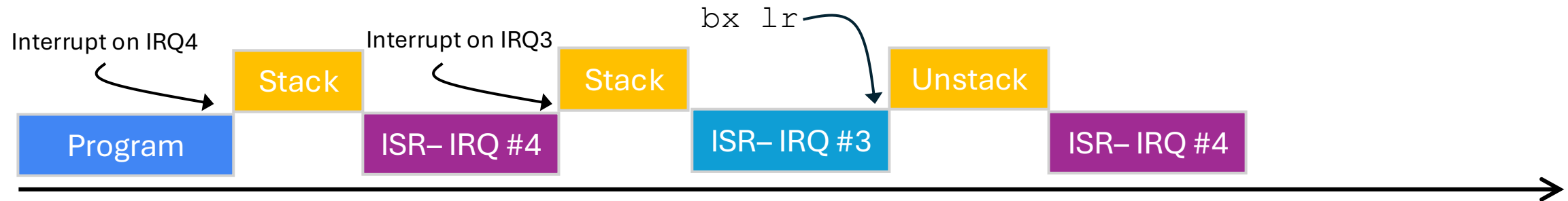
IRQ	
Priority Table	
0	8
1	10
1	5
3	3
4	6
Base Priority	
	6

Pending				
4				0
0	0	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
1	0	0	0	0

Example: Nested Interrupt



IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

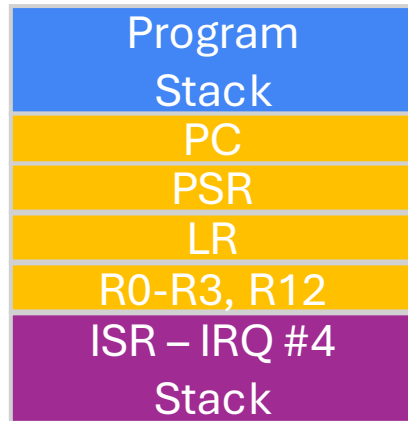
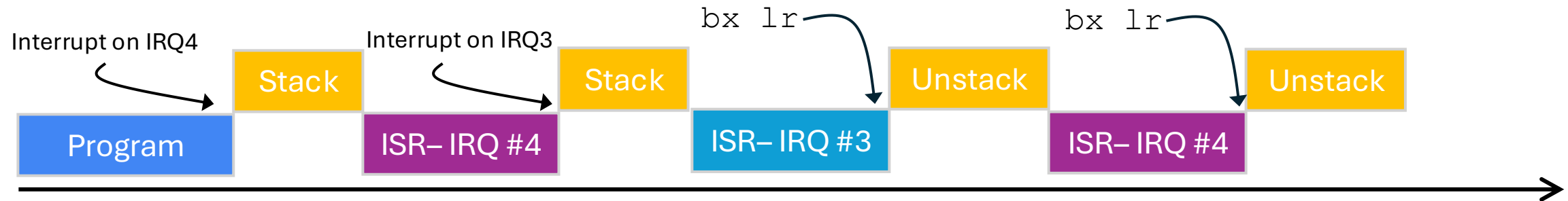
Base Priority	
	6

Pending				
4				0
0	0	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
1	0	0	0	0

Example: Nested Interrupt



IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

Base Priority	
	256

Pending				
4				0
0	0	0	0	0

Enabled				
4				0
1	1	1	1	1

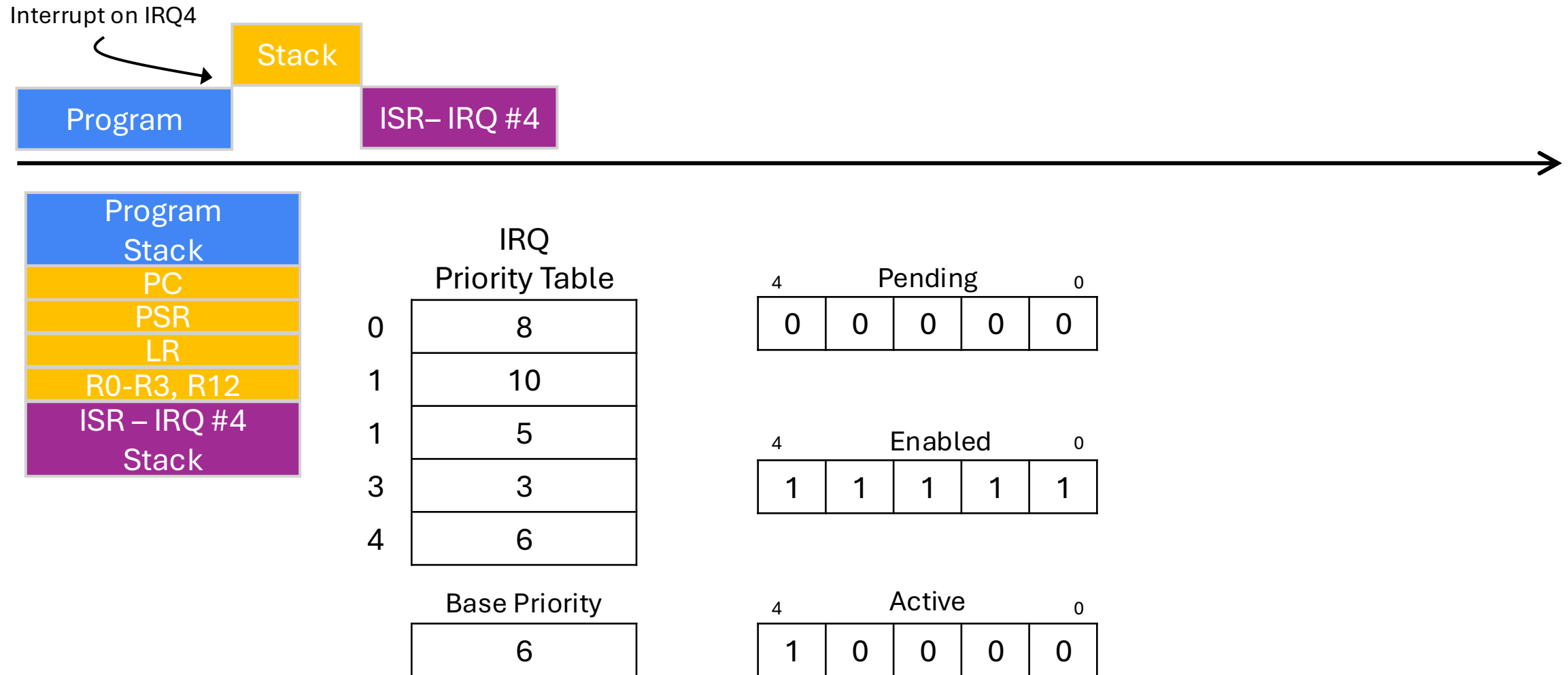
Active				
4				0
0	0	0	0	0

Tail Chaining

Same or lower important interrupt occurs while CPU is handling an interrupt of **same/higher priority**

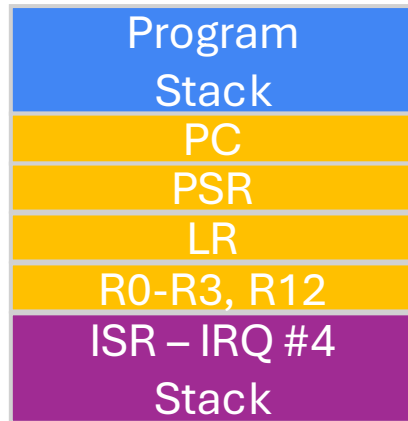
New exception will enter pending state....but will be **executed before register unstacking** (saving unnecessary unstacking/stacking operations)

Example: Nested Interrupt



Example: Nested Interrupt

Don't preempt



IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

Base Priority	
	6

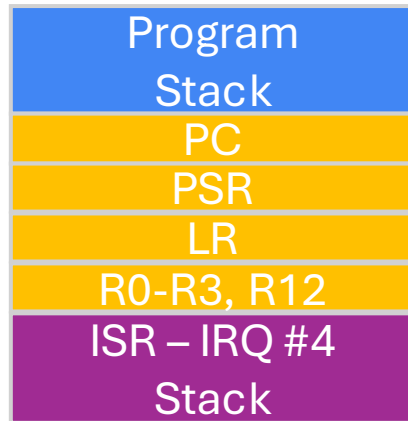
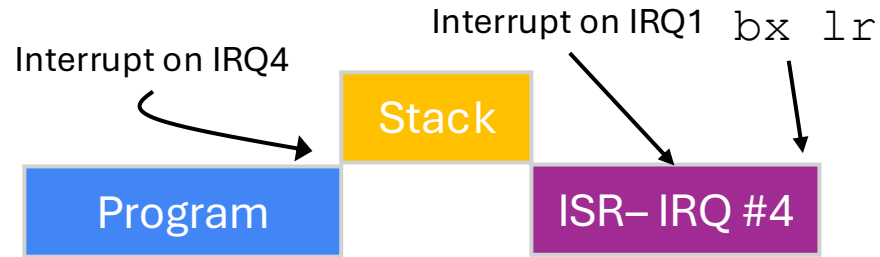
Pending				
4				0
0	0	0	1	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
1	0	0	0	0

Example: Nested Interrupt

Don't unstack



IRQ Priority Table

0	8
1	10
1	5
3	3
4	6

Base Priority

256

Pending

4					0
0	0	0	1	0	

Enabled

4					0
1	1	1	1	1	

Active

4					0
0	0	0	0	0	

Example: Nested Interrupt



Program
Stack
PC
PSR
LR
R0-R3, R12
ISR – IRQ #1
Stack

IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

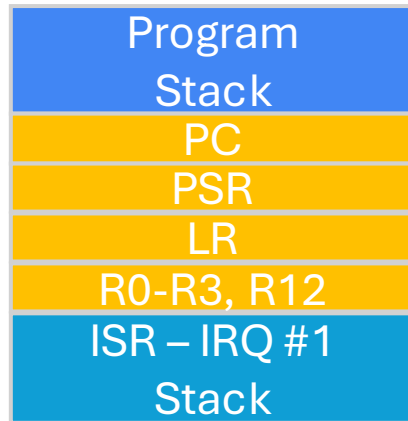
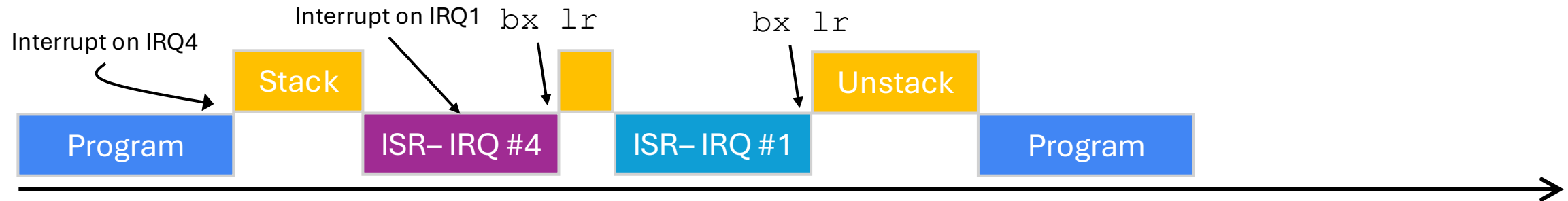
Base Priority	
	10

Pending				
4				0
0	0	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
0	0	0	1	0

Example: Nested Interrupt



IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

Base Priority	
	256

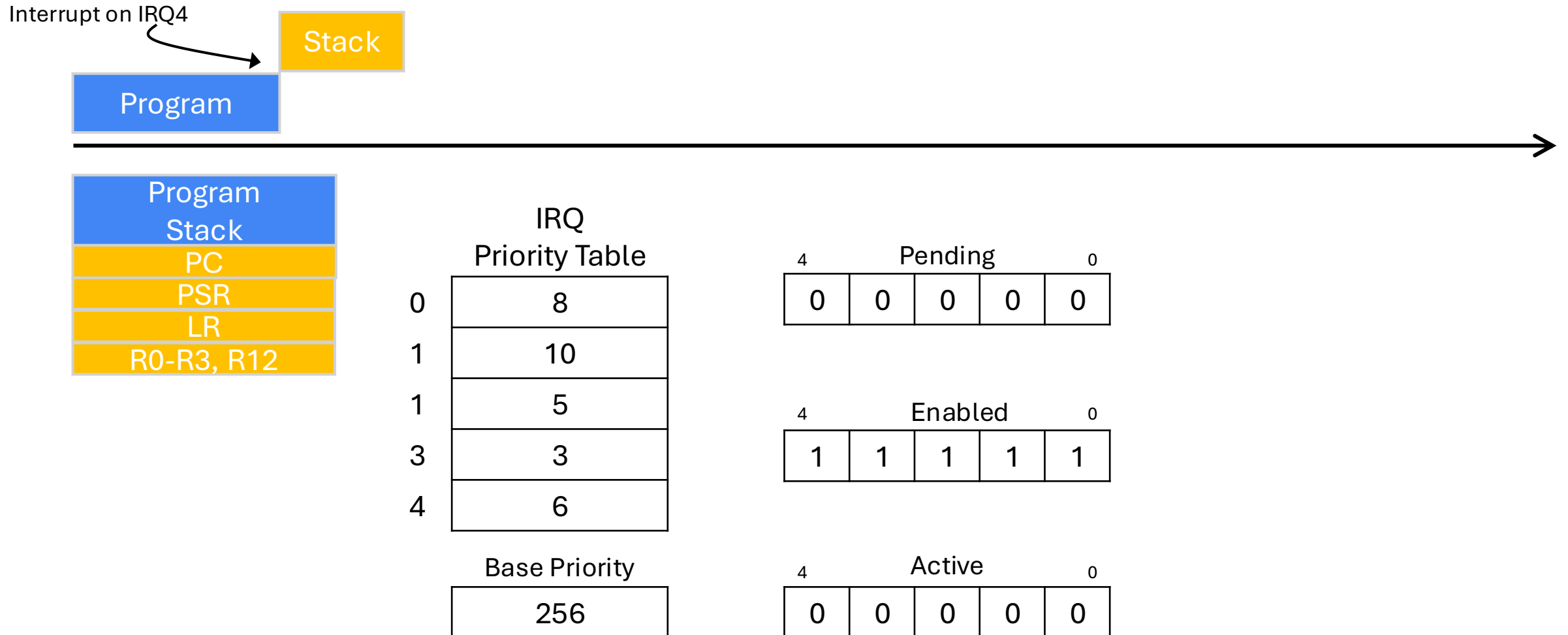
Pending				
4				0
0	0	0	0	0

Enabled				
4				0
1	1	1	1	1

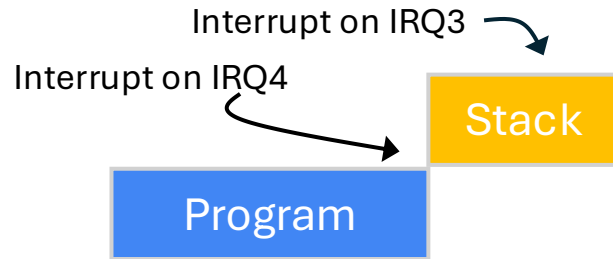
Active				
4				0
0	0	0	0	0

Example: Late Arrivals

What happens if an interrupt occurs while you are stacking?



Example: Late Arrivals



Program
Stack
PC
PSR
LR
R0-R3, R12

IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

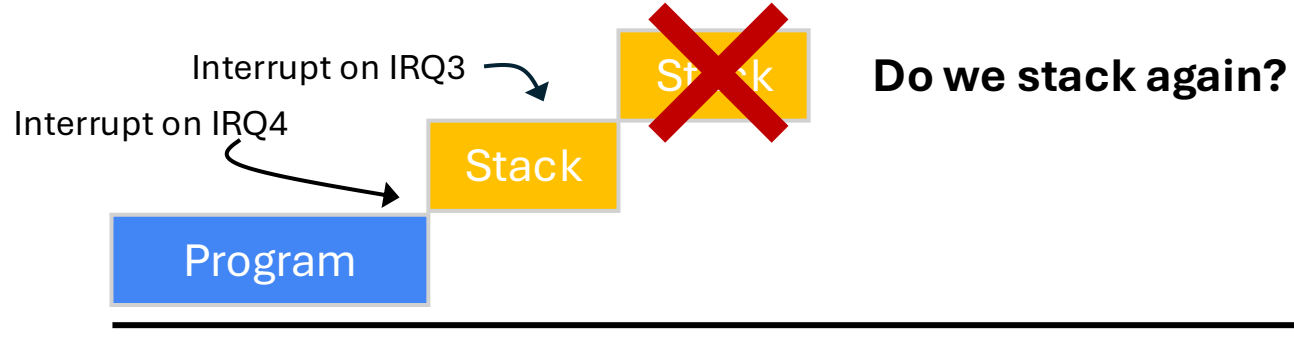
Base Priority
256

Pending				
4				0
1	1	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
0	0	0	0	0

Example: Late Arrivals



Program
Stack
PC
PSR
LR
R0-R3, R12

IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

Base Priority
6

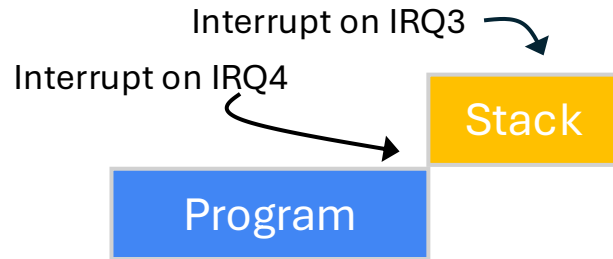
4	Pending			0
1	1	0	0	0

4	Enabled			0
1	1	1	1	1

4	Active			0
0	0	0	0	0

Example: Late Arrivals

Once stacking is complete,
find vector for the highest
priority pending



Program
Stack
PC
PSR
LR
R0-R3, R12

IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

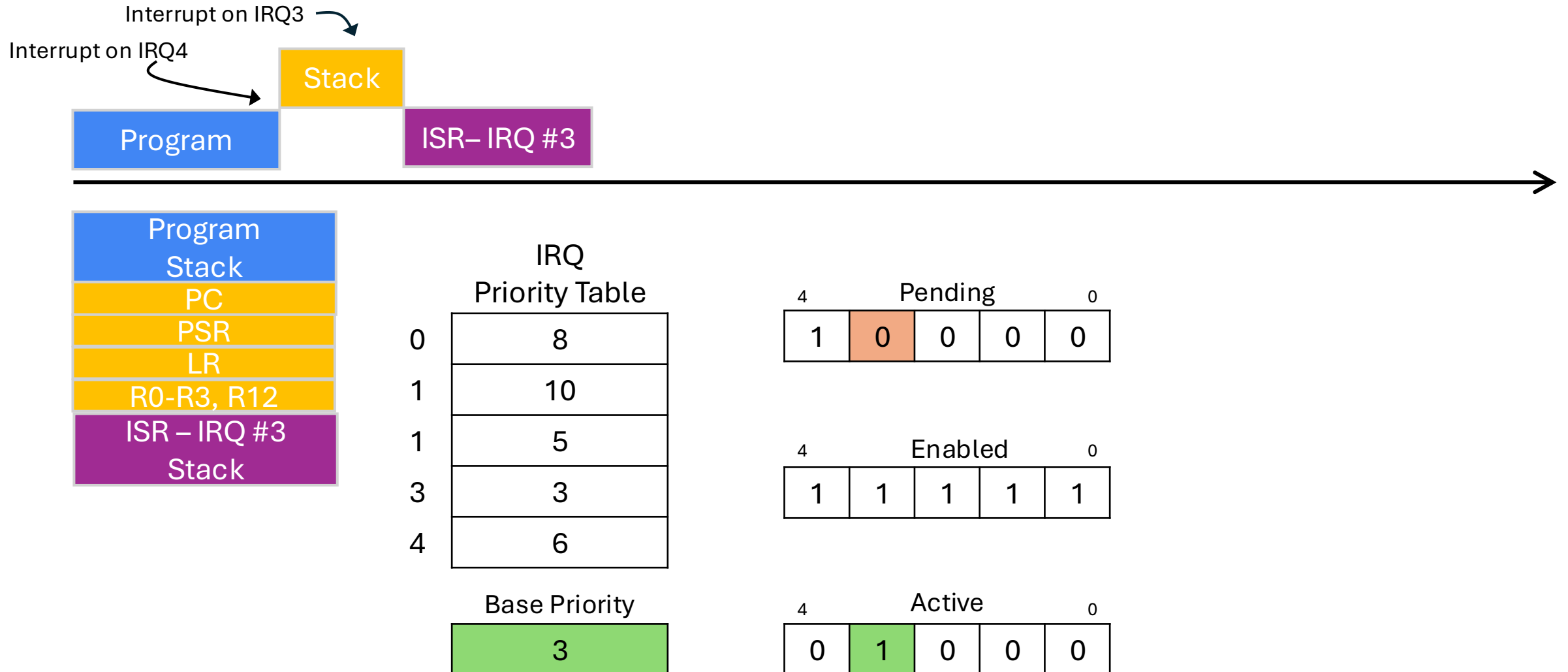
Base Priority
256

Pending				
4				0
1	1	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
1	0	0	0	0

Example: Late Arrivals



Example: Late Arrivals



Program
Stack
PC
PSR
LR
R0-R3, R12

IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

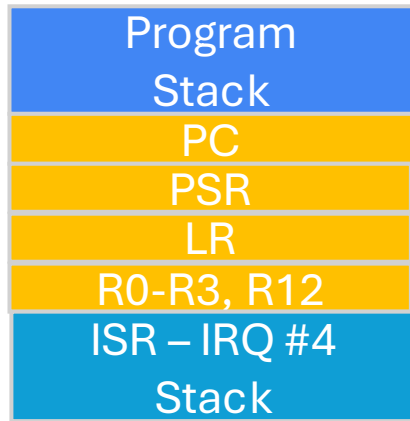
Base Priority
3

Pending				
4				0
1	0	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
0	1	0	0	0

Example: Late Arrivals



IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

Base Priority

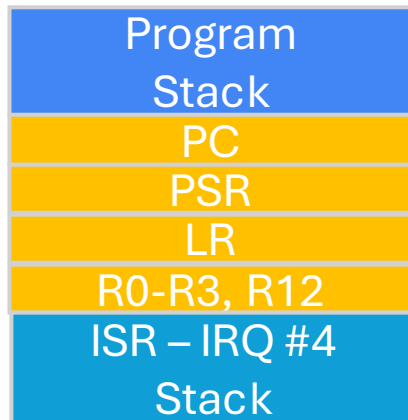
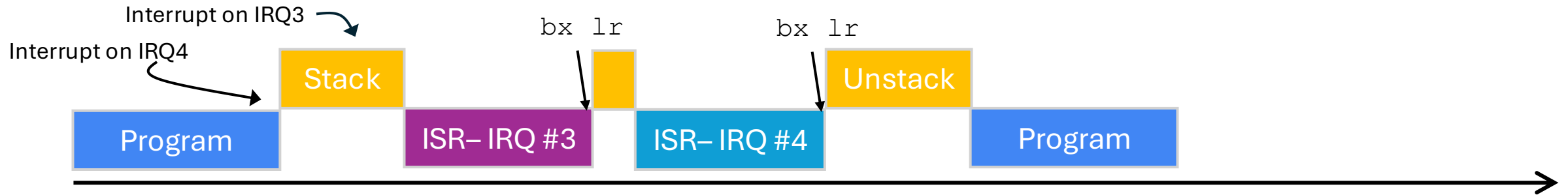
4

Pending				
4				0
0	0	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
1	0	0	0	0

Example: Late Arrivals



IRQ Priority Table	
0	8
1	10
1	5
3	3
4	6

Base Priority	
	256

Pending				
4				0
0	0	0	0	0

Enabled				
4				0
1	1	1	1	1

Active				
4				0
0	0	0	0	0

Acknowledgements

- These lecture slides were adapted from the University of Michigan's EECS 373 course (many thanks to Alanson Sample!)
- These lecture slides also leverage materials from the following books:
 - Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C
 - Yifeng Zhu
 - The Definitive Guide to ARM Cortex – M3 and Cortex – M4 Processors
 - Joseph Yiu
 - Introduction to ARM Cortex – M Microcontrollers, Embedded Systems
 - Jonathan W. Valvano